Distinction Graph Machines: Refine-or-Repair Runtime Learning

Anonymous

Abstract

We formalize online prediction with refinable information structures. At each time, a learner's runtime state induces a partition Π_t of a finite case space: repair changes the predictor on the current atoms, while refinement changes the partition itself by adding a binary distinction. We prove a repair-only lower bound: for any fixed partition Π, every learner whose predictions remain Π-measurable suffers expected regret at least $T(R_L(\Pi) - R_L(\Pi^\star))$ relative to a refined oracle $\Pi^\star$. Thus some runtime errors are not optimization errors but quotient errors. We characterize useful refinements by Bayes actions: for $\mathcal{F} \subseteq \mathcal{G}$, the refinement gap vanishes iff the $\mathcal{G}$-Bayes action is already $\mathcal{F}$-measurable. Under log loss this gap is $\mathbb{E}\, D_{\mathrm{KL}}(P(Y \mid \mathcal{G}) \| P(Y \mid \mathcal{F}))$, and a binary distinction has value $I(Y; U \mid \mathcal{F})$. We give finite-sample validation rules, distinction-complexity parameters, and realizable recovery guarantees for greedy refinement over finite distinction classes. Finally, we instantiate the theory as Distinction Graph Machines, a dynamic graph data structure whose nodes repair local memories and whose edges generate accepted distinctions. Controlled streams serve as separating examples: repair-only learners fail on missing-distinction tasks, while refine-and-repair learners recover the needed partition.

Contents

1 Introduction **1**

2 Online Prediction with Refinable Partitions **2**
2.1 Partitions, predictors, and risk . . . 2
2.2 Repair and refinement gaps . . . 3
2.3 Repair-only lower bound . . . 4
2.4 Sigma-algebra extension . . . 4
2.5 Minimal refinement . . . 5
2.5.1 The finite refinement rule . . . 5
2.5.2 The sigma-algebra form . . . 6

3 The Value of a Distinction **6**
3.1 Risk monotonicity . . . 6
3.2 Log loss . . . 6
3.3 Squared loss . . . 7
3.4 Minimal loss-sufficient refinement . . . 7
3.5 Finite empirical scores . . . 8
3.6 Distinction complexity . . . 8
3.7 Greedy recovery under realizability . . . 9
3.8 A finite-class oracle inequality . . . 10

4 DGM: Nodes Repair, Edges Distinguish **10**
4.1 Why a graph, not a tree? . . . 10
4.2 Hard-DGM state and invariants . . . 11
4.3 Soft-DGM edge-gated prediction . . . 12
4.4 Refine or repair on a graph . . . 14
4.5 Data-structure operations . . . 17
4.6 Canonical local memory . . . 18
4.7 From DGM to DGM-Embedding . . . 19

5 Guarantees **19**

6 Separating Examples and Sanity Checks **20**
6.1 Hidden-regime sequence prediction . . . 20
6.2 Repair-only ablation . . . 21
6.3 Controlled contextual XOR . . . 21
6.4 Synthetic next-token stream . . . 22

7 Related Work **22**

8 Conclusion **23**

A Appendix Roadmap **29**

B Limitations **29**

C Related Work **30**
C.1 Methods that repair without refining . . . 30
C.2 Methods that refine without local repair . . . 30
C.3 Methods that route without runtime concept creation . . . 31
C.4 DGM's position . . . 31

D DGM-Embedding: Architectural Extension for Language Models **31**
D.1 Hidden-state concepts . . . 32
D.2 Calibrated memory distribution . . . 32
D.3 Read before observe . . . 32
D.4 Sequential detached training . . . 33

E DGM Implementation Extensions **34**
E.1 Trees as a hard-routing special case . . . 34
E.2 DGM data-structure cost . . . 35
E.3 Ridge local memory . . . 35
E.4 Training modes for representation drift . . . 35
E.5 Non-canonical adapter extension . . . 36
E.6 Cost and systems notes . . . 36
E.7 Closed-form local repair . . . 37

F Additional Computational Checks **37**
F.1 Online image streams . . . 37
F.2 Frozen-embedding adapter check . . . 38
F.3 Projection-memory residual identity . . . 38

G Proof Sketches **38**

H From Distinctions to Projection Memory **44**
H.1 One-hot partition memory . . . 44
H.2 CPM conflict as missing distinction . . . 44
H.3 The hierarchy . . . 45

I Minimal Refinement Proofs **45**

J Value of Distinction Proofs **46**

K DGM Algorithm and Projection-Bridge Proofs **49**

1 Introduction

Runtime learning is usually phrased as an update problem: after a model is surprised, which memory, cache entry, adapter, or parameter should be changed? This paper studies the prior structural question. Before deciding how to update, a learner must decide whether the observed error is representable under its current information.

The central object of this paper is not a memory update, but the equivalence relation induced by the learner's runtime state. If two cases are assigned to the same runtime state, then any update that preserves that state space must treat them as indistinguishable. Repair modifies values on equivalence classes. Refinement modifies the equivalence relation itself.

Consider the smallest example. Let $\Omega = \{0, 1\}$, let P be uniform, and let the consequence be $Y = \omega$. If the current runtime representation has the trivial partition $\Pi = \{\{0, 1\}\}$, then under log loss the best repair-only predictor assigns probability $1/2$ to each label and pays $\log 2$ per round. The refined partition $\Pi^\star = \{\{0\}, \{1\}\}$ pays zero in the noiseless case. This gap is not caused by a bad optimizer, cache size, or local parameter value. It is caused by identifying two states that the target separates.

We formalize this as *online prediction with refinable partitions.* A learner maintains a state

$$S_t = (\Pi_t, a_t),$$

where Π_t is a finite partition of the case space and $a_t : \Pi_t \to \mathsf{A}$ is a predictor on its atoms. On each round, the learner observes only the current atom containing the sampled case, predicts, observes the consequence, and then chooses either repair or refinement. A repair changes a_t while keeping $\Pi_{t+1} = \Pi_t$. A refinement chooses an admissible binary distinction U and replaces the partition by the join $\Pi_{t+1} = \Pi_t \vee U$.

This model makes a lower bound immediate and useful. For any fixed partition Π, every repair-only learner whose predictions remain Π-measurable has expected per-round loss at least the Bayes risk $R_L(\Pi)$. Relative to an oracle using a refined partition $\Pi^\star$, its expected regret is at least

$$T\left(R_L(\Pi) - R_L(\Pi^\star)\right).$$

Thus, whenever the refinement gap is positive, repair-only learning has linear regret against a refined oracle. Caches, local memories, fast weights, nearest-neighbor writes, and test-time optimizers fall under this statement only when their updates preserve the relevant partition. Methods that change the representation are refinements in this model, not repairs.

The minimal escape from this lower bound is also canonical. Adding a binary distinction U replaces the current partition by

$$\Pi \vee U = \{B \cap U^{-1}(0), B \cap U^{-1}(1) : B \in \Pi\} \setminus \{\varnothing\}.$$

This operation cuts exactly the atoms crossed by U and leaves all other atoms unchanged. A distinction is therefore not merely an auxiliary feature; it changes the information structure on which future predictions are allowed to depend.

Not every new bit is worth adding. A refinement should be accepted only if it changes the best achievable decision. We characterize this condition in Bayes-risk terms: if $\mathcal{F} \subseteq \mathcal{G}$ and the $\mathcal{G}$-Bayes action is unique, then $R_L(\mathcal{F}) - R_L(\mathcal{G}) = 0$ iff that refined Bayes action already has an $\mathcal{F}$-measurable version. Under log loss, the gap becomes an expected conditional KL divergence, and a binary distinction has value $I(Y; U \mid \mathcal{F})$. This gives a consequence-based split score: accept distinctions because they change outcome distributions, not because they are geometrically novel.

Distinction Graph Machines (DGM) are the data-structure realization of this abstract model. The formal object is Hard-DGM: fixed anchors and accepted binary edges generate a partition, node memories repair predictions on the resulting states, and refinement adds a new binary generator. The practical implementation is Soft-DGM: centroids, sparse retrieval, and differentiable edge gates approximate the hard route. The formal guarantees apply directly to Hard-DGM and to Soft-DGM only under candidate-recall and margin conditions.

Contributions. We make six contributions.

- **Model.** We introduce online prediction with refinable information partitions, where repair preserves Π_t and refinement replaces Π_t by $\Pi_t \vee U$.
- **Lower bound.** We prove that any repair-only learner suffers linear regret against a refined oracle whenever $R_L(\Pi) > R_L(\Pi^\star)$.
- **Characterization.** We show that a refinement has positive value exactly when it changes the Bayes action; under log loss, the value of a binary distinction is $I(Y; U \mid \mathcal{F})$.
- **Algorithmic guarantees.** We give statistically guarded split tests, distinction-complexity parameters, realizable greedy recovery guarantees, and a finite-class oracle inequality.
- **Data structure.** We implement refinable partitions as DGM, where graph edges are binary generators and node memories implement repair.
- **Separating examples.** We give controlled streams showing that repair-only learners fail on missing-distinction tasks while refine-and-repair learners recover the needed partition. These are mechanism checks, not claims of language-model-scale competitiveness.

Section 2 defines refinable online prediction and proves the repair-only lower bound. Section 2.5 gives the minimal join operation. Section 3 characterizes useful distinctions and gives finite-sample refinement guarantees. Section 4 defines DGM as the data-structure implementation. The remaining sections give scoped guarantees, separating examples, and related work; DGM-Embedding and soft implementation details are in the appendix.

2 Online Prediction with Refinable Partitions

We start with a finite model. The measurable-space notation used later is the direct sigma-algebra generalization, but the computational object is a partition. A runtime state declares which cases are equivalent; a repair changes predictions on equivalence classes, while a refinement changes the equivalence relation.

2.1 Partitions, predictors, and risk

Let Ω be a finite case space, let P be a distribution on Ω, let $Y : \Omega \to \mathcal{Y}$ be the consequence variable, let A be an action space, and let $L : \mathcal{Y} \times \mathsf{A} \to \mathbb{R}$ be a loss. A finite partition

$$\Pi = \{B_1, \ldots, B_m\}$$

induces the equivalence relation $\omega \sim_\Pi \omega'$ iff ω, ω' lie in the same atom $B \in \Pi$. A predictor under Π is a map $a : \Pi \to \mathsf{A}$, equivalently a function $f : \Omega \to \mathsf{A}$ that is constant on every atom:

$$\mathsf{Pred}(\Pi) = \{f : \Omega \to \mathsf{A} : \omega \sim_\Pi \omega' \Rightarrow f(\omega) = f(\omega')\}.$$

The Bayes risk achievable under partition Π is

$$R_L(\Pi) = \inf_{f \in \mathsf{Pred}(\Pi)} \mathbb{E}[L(Y, f)] = \sum_{B \in \Pi} P(B) \inf_{a \in \mathsf{A}} \mathbb{E}[L(Y, a) \mid \omega \in B], \tag{1}$$

where atoms with $P(B) = 0$ may be assigned arbitrary actions. Thus $R_L(\Pi)$ is the best possible risk after the representation has committed to the equivalence relation $\sim_\Pi$.

Definition 2.1 (Refinable online prediction problem)**.** *A refinable online prediction problem is a tuple*

$$\mathfrak{P} = (\Omega, \mathcal{Y}, \mathsf{A}, P, L, \mathcal{U}, c),$$

where Ω is finite, $Y : \Omega \to \mathcal{Y}$ is the consequence variable, P is the sampling distribution, $L : \mathcal{Y} \times \mathsf{A} \to [0, 1]$ is a bounded loss, $\mathcal{U} \subseteq \{0, 1\}^\Omega$ is a class of admissible binary distinctions, and $c : \mathcal{U} \to \mathbb{R}_+$ assigns refinement costs.

At round t, the learner maintains $S_t = (\Pi_t, a_t)$ with $a_t : \Pi_t \to \mathsf{A}$. A case $\omega_t \sim P$ is drawn, the learner observes only the atom $B_t \in \Pi_t$ containing ω_t, predicts $a_t(B_t)$, observes $Y_t = Y(\omega_t)$, and then chooses one of two updates:

$$\begin{aligned} \text{Repair}: &\quad \Pi_{t+1} = \Pi_t, \quad a_{t+1} \text{ may change}, \\ \text{Refine}: &\quad \Pi_{t+1} = \Pi_t \vee U_t \text{ for some } U_t \in \mathcal{U}. \end{aligned}$$

For a binary distinction $U : \Omega \to \{0, 1\}$,

$$\Pi \vee U = \{B \cap U^{-1}(0), B \cap U^{-1}(1) : B \in \Pi\} \setminus \{\varnothing\}. \tag{2}$$

The cumulative objective is prediction loss plus refinement cost,

$$\mathcal{L}_T = \sum_{t=1}^{T} L(Y_t, a_t(B_t)) + \lambda \sum_{t:\text{Refine}} c(U_t).$$

The partition formulation also covers information maps. If a runtime representation is $I_t : \Omega \to \mathcal{Z}_t$, its induced partition is the set of nonempty fibers $I_t^{-1}(z)$. Repair preserves these fibers. Refinement splits at least one fiber.

2.2 Repair and refinement gaps

For a concrete predictor $f \in \mathsf{Pred}(\Pi)$ and a candidate refined partition $\Pi^\star$ that refines Π, define

$$\Delta_{\text{repair}}(f, \Pi) = R_L(f) - R_L(\Pi), \tag{3}$$

$$\Delta_{\text{refine}}(\Pi, \Pi^\star) = R_L(\Pi) - R_L(\Pi^\star). \tag{4}$$

The first gap is the residual error of the current predictor at fixed equivalence relation. The second is the value of changing the equivalence relation.

Theorem 2.2 (Runtime error decomposition)**.** *Let $f_t \in \mathsf{Pred}(\Pi_t)$ and let $\Pi^\star$ refine Π_t. Then*

$$R_L(f_t) - R_L(\Pi^\star) = \Delta_{\text{repair}}(f_t, \Pi_t) + \Delta_{\text{refine}}(\Pi_t, \Pi^\star). \tag{5}$$

Both terms are nonnegative.

Proof. Add and subtract $R_L(\Pi_t)$. The repair gap is nonnegative because f_t is feasible for the infimum defining $R_L(\Pi_t)$. The refinement gap is nonnegative because every Π_t-measurable predictor is also measurable with respect to any refinement $\Pi^\star$. □

2.3 Repair-only lower bound

The pointwise contradiction inside a cell becomes a sequential lower bound. If the current partition collapses states that a refined oracle separates, no repair-only learner can avoid paying the corresponding Bayes-risk gap every round.

Theorem 2.3 (Repair-only linear regret)**.** *Let Π be a fixed partition and let $\Pi^\star$ be a refinement. Consider any learner whose round-t prediction $\hat{f}_t$ is always in $\mathsf{Pred}(\Pi)$, possibly chosen adaptively from all previous observations. Then, for iid $\omega_t \sim P$,*

$$\mathbb{E}\left[\sum_{t=1}^{T} L(Y_t, \hat{f}_t(\omega_t))\right] \geq T R_L(\Pi).$$

Consequently, relative to an oracle using $\Pi^\star$,

$$\mathbb{E}[\mathrm{Regret}_T] \geq T\left(R_L(\Pi) - R_L(\Pi^\star)\right).$$

If $R_L(\Pi) > R_L(\Pi^\star)$, every repair-only learner suffers $\Omega(T)$ expected regret against the refined oracle.

Proof. Condition on the learner's history before round t. The chosen predictor $\hat{f}_t$ is Π-measurable, so by definition of $R_L(\Pi)$,

$$\mathbb{E}[L(Y_t, \hat{f}_t(\omega_t)) \mid \text{history}] \geq R_L(\Pi).$$

Taking expectation and summing over t gives the first inequality. The $\Pi^\star$ oracle achieves $R_L(\Pi^\star)$ per round by using the Bayes action on each atom of $\Pi^\star$, so subtracting its cumulative risk gives the regret bound. □

Example 2.4 (Two-state separation)**.** *Let $\Omega = \{0,1\}$, $P(0) = P(1) = 1/2$, and $Y = \omega$. Under the trivial partition $\Pi = \{\{0,1\}\}$, the best log-loss predictor is $q(Y=0) = q(Y=1) = 1/2$, so $R_{\log}(\Pi) = \log 2$. The refined partition $\Pi^\star = \{\{0\},\{1\}\}$ has $R_{\log}(\Pi^\star) = 0$ in the noiseless case. Therefore any repair-only learner has expected regret $T \log 2$ against the refined oracle. This example is not about neural networks, caches, or trees. It is about equivalence relations: a learner that identifies two states cannot assign them two different consequences.*

Corollary 2.5 (Contradiction requires distinction)**.** *Let Π be a partition and let $y : \Omega \to \mathsf{A}$ be a required deterministic output. If ω, ω' lie in the same atom of Π but $y(\omega) \neq y(\omega')$, then no predictor in $\mathsf{Pred}(\Pi)$ satisfies both requirements.*

Proof. Every $f \in \mathsf{Pred}(\Pi)$ is constant on the atom containing ω and ω', so $f(\omega) = f(\omega')$. Satisfying both requirements would imply $y(\omega) = y(\omega')$, a contradiction. □

2.4 Sigma-algebra extension

The finite partition model is the main computational model. The sigma-algebra notation is the measure-theoretic extension used for value identities and language-model interpretations. On a probability space $(\Omega, \mathcal{E}, \mathbb{P})$, an information structure is a sub-sigma-algebra $\mathcal{F} \subseteq \mathcal{E}$, and

$$\mathsf{Pred}(\mathcal{F}) = \{a : \Omega \to \mathsf{A} : a \text{ is } \mathcal{F}\text{-measurable}\}, \qquad R_L(\mathcal{F}) = \inf_{a \in \mathsf{Pred}(\mathcal{F})} \mathbb{E}[L(Y,a)].$$

For a finite partition Π, this recovers the previous definitions by taking $\mathcal{F} = \sigma(\Pi)$.

When the infimum is attained, write $a^\star_{\mathcal{F}}$ for a Bayes action under $\mathcal{F}$. A richer information structure is useful only when its optimal action cannot already be expressed using the old information.

Theorem 2.6 (Loss-relevant refinement)**.** *Let $\mathcal{F} \subseteq \mathcal{G} \subseteq \mathcal{E}$. Assume Bayes actions exist under $\mathcal{F}$ and $\mathcal{G}$, and that the $\mathcal{G}$-Bayes action is almost surely unique. Then*

$$R_L(\mathcal{F}) - R_L(\mathcal{G}) = 0$$

if and only if $a^\star_{\mathcal{G}}$ has an $\mathcal{F}$-measurable version.

Proof. If $a^\star_{\mathcal{G}}$ is $\mathcal{F}$-measurable, then it is feasible for $R_L(\mathcal{F})$, so

$$R_L(\mathcal{F}) \leq \mathbb{E}[L(Y, a^\star_{\mathcal{G}})] = R_L(\mathcal{G}).$$

Monotonicity under $\mathcal{F} \subseteq \mathcal{G}$ gives the reverse inequality. Conversely, if the risks are equal and $a^\star_{\mathcal{F}}$ is an $\mathcal{F}$-Bayes action, then $a^\star_{\mathcal{F}}$ is also feasible under $\mathcal{G}$ and achieves $R_L(\mathcal{G})$. By almost sure uniqueness under $\mathcal{G}$, $a^\star_{\mathcal{F}} = a^\star_{\mathcal{G}}$ almost surely. Hence $a^\star_{\mathcal{G}}$ has an $\mathcal{F}$-measurable version. □

2.5 Minimal refinement

Once a contradiction shows that the current atom is too coarse, the next question is not how to grow a large model. It is which information must be added. The primitive operation in this paper is minimal refinement: given an information structure and evidence that a new event or variable must be observable, add that observability and nothing else. We state the finite partition theorem first, then the sigma-algebra form. The latter is the standard measure-theoretic join; the former is its finite special case [Bil95, Kal02].

2.5.1 The finite refinement rule

Let $\mathcal{P}$ be a finite partition and let $D \subseteq \Omega$ be a new binary distinction event. Define

$$\begin{aligned} \mathcal{P}_D = &\{B \cap D : B \in \mathcal{P}, B \cap D \neq \varnothing\} \\ &\cup \{B \cap D^c : B \in \mathcal{P}, B \cap D^c \neq \varnothing\}. \end{aligned} \tag{6}$$

This keeps exactly the nonempty pieces obtained by cutting each old atom by D.

Theorem 2.7 (Minimal finite refinement)**.** *Let $\mathcal{P}$ be a finite partition of Ω and let $D \subseteq \Omega$. The partition $\mathcal{P}_D$ is the unique minimal partition that refines $\mathcal{P}$ and makes D measurable. More precisely, if $\mathcal{Q}$ refines $\mathcal{P}$ and D is $\mathcal{Q}$-measurable, then $\mathcal{Q}$ refines $\mathcal{P}_D$.*

Proof. Each old atom $B \in \mathcal{P}$ is replaced by the nonempty sets among $B \cap D$ and $B \cap D^c$, so the resulting sets are disjoint, cover Ω, and each lies inside an atom of $\mathcal{P}$. Hence $\mathcal{P}_D$ is a refinement of $\mathcal{P}$. Moreover,

$$D = \bigcup_{B \in \mathcal{P}} (B \cap D),$$

so D is a union of $\mathcal{P}_D$-atoms. Now let $\mathcal{Q}$ refine $\mathcal{P}$ and make D measurable. For every $Q \in \mathcal{Q}$, there is $B \in \mathcal{P}$ with $Q \subseteq B$, and measurability of D with respect to $\mathcal{Q}$ implies either $Q \subseteq D$ or $Q \subseteq D^c$. Thus Q is contained in either $B \cap D$ or $B \cap D^c$, i.e., in an atom of $\mathcal{P}_D$. Therefore $\mathcal{Q}$ refines $\mathcal{P}_D$. □

The theorem gives a canonical update. The evidence determines the least refined representation: atoms untouched by D remain unchanged, while atoms crossed by D split into the two pieces that the evidence distinguishes.

2.5.2 The sigma-algebra form

For general spaces, the same operation is the join of sigma-algebras. If the current information is $\mathcal{F}_t$ and the evidence is a variable U_t, the minimal updated information is

$$\mathcal{F}_{t+1} = \mathcal{F}_t \vee \sigma(U_t). \tag{7}$$

Theorem 2.8 (Minimal sigma-algebra refinement)**.** *Let $\mathcal{F} \subseteq \mathcal{E}$ be a sigma-algebra and let U be a random variable. The join $\mathcal{F} \vee \sigma(U)$ is the smallest sigma-algebra $\mathcal{G}$ such that $\mathcal{F} \subseteq \mathcal{G}$ and U is $\mathcal{G}$-measurable.*

Proof. By definition, $\mathcal{F} \vee \sigma(U) = \sigma(\mathcal{F} \cup \sigma(U))$, so it contains $\mathcal{F}$ and makes U measurable. If another sigma-algebra $\mathcal{G}$ contains $\mathcal{F}$ and makes U measurable, then $\mathcal{F} \cup \sigma(U) \subseteq \mathcal{G}$. Since $\mathcal{G}$ is a sigma-algebra, $\sigma(\mathcal{F} \cup \sigma(U)) \subseteq \mathcal{G}$. □

Thus the same object governs finite decision-tree splits and measurable-space refinement. A distinction is not an auxiliary feature added after learning; it is a change in the sigma-algebra of what the learner may condition on. The obstruction in Section 2 says when such a change is necessary; minimal refinement says how little information must be added.

3 The Value of a Distinction

Minimal refinement says how to add a distinction; it does not license every possible distinction. The value question is task-dependent: when does the new distinction actually change the best achievable prediction? We measure this by the decrease in optimal risk after the information structure is refined, in the same decision-theoretic sense used in classical comparisons of statistical experiments and value of information [Bla53, Ber85].

3.1 Risk monotonicity

The first fact is purely order-theoretic. If $\mathcal{G}$ contains all events in $\mathcal{F}$, then every $\mathcal{F}$-measurable predictor is also $\mathcal{G}$-measurable. The decision class has only grown; this is the Bayes-risk analogue of Blackwell's monotonicity of decision value under more informative experiments [Bla53].

Theorem 3.1 (Bayes-risk monotonicity)**.** *If $\mathcal{F} \subseteq \mathcal{G}$, then for every loss L for which both risks are well-defined,*

$$R_L(\mathcal{G}) \leq R_L(\mathcal{F}).$$

This is a population statement. It says that information cannot hurt the best possible decision rule. It does not say that a finite implementation should refine without restraint. Statistical estimation, finite samples, and model selection can overfit; those are algorithmic questions, not failures of the information order. This is why the DGM algorithm later uses penalties, thresholds, and sample-split scoring before committing a new concept.

3.2 Log loss

For a finite target variable Y, log loss asks the learner to output a conditional distribution. The optimal prediction under $\mathcal{F}$ is the conditional law of Y given $\mathcal{F}$, and the optimal risk is conditional entropy, a standard identity for logarithmic scoring rules and information theory [CT06, GR07].

Theorem 3.2 (Log-loss value)**.** *Let Y be finite and let the loss be $-\log q(Y)$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = I(Y; \mathcal{G} \mid \mathcal{F}).$$

Here $I(Y; \mathcal{G} \mid \mathcal{F})$ denotes $H(Y \mid \mathcal{F}) - H(Y \mid \mathcal{G})$. Writing

$$p_{\mathcal{F}}(y) = \mathbb{P}(Y = y \mid \mathcal{F}), \qquad p_{\mathcal{G}}(y) = \mathbb{P}(Y = y \mid \mathcal{G}),$$

the same identity has the conditional-KL form

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = \mathbb{E}\left[D_{\mathrm{KL}}(p_{\mathcal{G}}(\cdot) \| p_{\mathcal{F}}(\cdot))\right]. \tag{8}$$

Thus, for classification and language modeling, a refinement is useful exactly when it changes the conditional consequence distribution with positive probability. If the new bit does not tell us anything about the next consequence beyond the old information, it is a gratuitous distinction, even if it separates inputs.

Corollary 3.3 (Binary distinction value)**.** *Let $U \in \{0, 1\}$ be a candidate distinction and let $\mathcal{G} = \mathcal{F} \vee \sigma(U)$. Under log loss,*

$$\begin{aligned} R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) &= I(Y; U \mid \mathcal{F}) \\ &= \mathbb{E}\left[\sum_{u \in \{0,1\}} \mathbb{P}(U = u \mid \mathcal{F}) D_{\mathrm{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{F}, U = u) \| \mathbb{P}(Y \in \cdot \mid \mathcal{F}))\right]. \end{aligned}$$

3.3 Squared loss

For $Y \in L^2(\Omega; \mathbb{R}^d)$ and squared loss, the optimal predictor under $\mathcal{F}$ is $\mathbb{E}[Y \mid \mathcal{F}]$, the L^2 projection onto the closed subspace of $\mathcal{F}$-measurable random variables [Bil95, Kal02]. Refinement helps precisely when it changes the conditional mean.

Theorem 3.4 (Squared-loss value)**.** *Let $Y \in L^2(\Omega; \mathbb{R}^d)$ and let $L(Y, a) = \|Y - a\|_2^2$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_2(\mathcal{F}) - R_2(\mathcal{G}) = \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].$$

The identity is the Hilbert-space analogue of the log-loss result. A distinction has squared-loss value when the cells it creates have different conditional means. In regression language, refinement is worth paying for when it separates consequences, not merely when it separates inputs.

3.4 Minimal loss-sufficient refinement

Minimal refinement in Section 2.5 adds the least information needed to observe a proposed variable. The loss-sufficient question is slightly different: among the refinements inside $\mathcal{G}$, what is the least information needed to attain the $\mathcal{G}$-Bayes risk?

Theorem 3.5 (Minimal loss-sufficient refinement)**.** *Let $\mathcal{F} \subseteq \mathcal{G}$ and assume the $\mathcal{G}$-Bayes action $a^\star_{\mathcal{G}}$ exists and is almost surely unique. Up to null sets, the smallest sigma-algebra $\mathcal{H}$ satisfying*

$$\mathcal{F} \subseteq \mathcal{H} \subseteq \mathcal{G}, \qquad R_L(\mathcal{H}) = R_L(\mathcal{G})$$

is

$$\mathcal{F} \vee \sigma(a^\star_{\mathcal{G}}).$$

For finite Y under log loss, the corresponding smallest sufficient refinement is

$$\mathcal{F} \vee \sigma(p_{\mathcal{G}}), \qquad p_{\mathcal{G}}(\cdot) = \mathbb{P}(Y \in \cdot \mid \mathcal{G}).$$

Proof. The action $a^\star_{\mathcal{G}}$ is measurable with respect to $\mathcal{F} \vee \sigma(a^\star_{\mathcal{G}})$, so this refinement attains $R_L(\mathcal{G})$. If any $\mathcal{H}$ with $\mathcal{F} \subseteq \mathcal{H} \subseteq \mathcal{G}$ also attains $R_L(\mathcal{G})$, Theorem 2.6 applied to $\mathcal{H} \subseteq \mathcal{G}$ implies that $a^\star_{\mathcal{G}}$ has an $\mathcal{H}$-measurable version. Hence $\mathcal{F} \vee \sigma(a^\star_{\mathcal{G}}) \subseteq \mathcal{H}$ up to null sets. Under log loss, the Bayes action is the conditional law $p_{\mathcal{G}}$, giving the final statement. □

3.5 Finite empirical scores

The population identities lead directly to standard empirical split scores: information gain for classification and variance reduction for regression trees [BFOS84, DH00]. Suppose an atom B contains samples (x_j, y_j) and a candidate distinction $g : B \to \{0, 1\}$ splits it into B_0 and B_1.

For classification, let $\hat{p}_B$, $\hat{p}_{B_0}$, and $\hat{p}_{B_1}$ be empirical label distributions. The empirical log-loss improvement is

$$\Delta_{\log}(B, g) = |B|\, H(\hat{p}_B) - |B_0|\, H(\hat{p}_{B_0}) - |B_1|\, H(\hat{p}_{B_1}).$$

Equivalently,

$$\Delta_{\log}(B, g) = |B|\, \widehat{I}(Y; g \mid B).$$

The normalized empirical edge score is therefore

$$\widehat{I}(Y; g \mid B) = H(\hat{p}_B) - \sum_{b \in \{0,1\}} \frac{|B_b|}{|B|} H(\hat{p}_{B_b}).$$

For regression, define

$$\hat{\mu}_B = \frac{1}{|B|} \sum_{j \in B} y_j, \qquad \mathrm{SSE}(B) = \sum_{j \in B} \|y_j - \hat{\mu}_B\|_2^2 .$$

The empirical squared-loss improvement is

$$\Delta_2(B, g) = \mathrm{SSE}(B) - \mathrm{SSE}(B_0) - \mathrm{SSE}(B_1).$$

It also has the between-cell form

$$\Delta_2(B, g) = |B_0|\, \|\hat{\mu}_{B_0} - \hat{\mu}_B\|_2^2 + |B_1|\, \|\hat{\mu}_{B_1} - \hat{\mu}_B\|_2^2 .$$

These scores implement the same principle as the population theory: split only when the proposed distinction has measurable consequence value. The DGM algorithm below turns this population rule into a guarded empirical decision.

3.6 Distinction complexity

The refinement gap measures the value of a target information structure. A complementary question is how many admissible distinctions are needed to reach it. Let Π be an initial partition, let $\Pi^\star$ be a target refinement, and let $\mathcal{U} \subseteq \{0, 1\}^\Omega$ be a distinction class. For $\epsilon \geq 0$, define the loss-dependent distinction complexity

$$d_L^\epsilon(\Pi, \Pi^\star; \mathcal{U}) = \min \{m : \exists U_1, \ldots, U_m \in \mathcal{U} \text{ such that } R_L(\Pi \vee U_1 \vee \cdots \vee U_m) \leq R_L(\Pi^\star) + \epsilon\}, \quad (9)$$

with value $+\infty$ if no such sequence exists. Repair-only learning is the case $m = 0$. Decision trees, graph refinements, and prototype split systems differ partly in the distinction classes and ordering constraints they impose. DGM corresponds to a graph-generated class of local binary predicates proposed from confused runtime states.

3.7 Greedy recovery under realizability

The next theorem is a clean realizable upper bound for greedy refinement. It is intentionally stated for the abstract partition model rather than for Soft-DGM retrieval.

Definition 3.6 (Target-respecting distinction)**.** *Let $\Pi^\star$ be a target partition. A distinction $U : \Omega \to \{0,1\}$ is $\Pi^\star$-respecting if it does not split any atom of $\Pi^\star$: for every $C \in \Pi^\star$, either $C \subseteq U^{-1}(0)$ or $C \subseteq U^{-1}(1)$.*

For an atom $B \in \Pi$, write $\Pi^\star|_B = \{C \in \Pi^\star : C \cap B \neq \varnothing\}$. The atom B is impure if $|\Pi^\star|_B| > 1$. For a candidate distinction U, define its local value on B by

$$V_L(B, U) = R_L(\{B\}) - R_L(\{B\} \vee U)\,,$$

where the risks are conditional on $\omega \in B$. Under log loss, $V_{\log}(B, U) = I(Y; U \mid \omega \in B)$.

Theorem 3.7 (Greedy recovery under target-respecting splits)**.** *Let $\Pi^\star$ have K atoms and suppose the current partition is initially a coarsening of $\Pi^\star$. Assume that every impure current atom B admits a $\Pi^\star$-respecting distinction $U \in \mathcal{U}$ such that U splits $\Pi^\star|_B$ into two nonempty groups and*

$$V_L(B, U) \geq \gamma.$$

Consider a greedy refinement algorithm whose candidate set contains at least one such distinction for every impure atom, whose accepted candidates are $\Pi^\star$-respecting, and whose empirical local values satisfy

$$\left|\widehat{V}_L(B, U) - V_L(B, U)\right| \leq \gamma/4$$

for all candidates it scores. If the algorithm accepts a split whenever $\widehat{V}_L(B, U) > \gamma/2$, then it accepts a useful split in every impure atom it visits and recovers $\Pi^\star$ after at most $K - 1$ accepted splits.

Proof. Because every accepted split is $\Pi^\star$-respecting, the current partition remains a coarsening of $\Pi^\star$ after each accepted refinement. A candidate with true value at least γ has empirical value at least $3\gamma/4$, so it exceeds the $\gamma/2$ threshold and is accepted when proposed. Since the candidate also splits $\Pi^\star|_B$ into two nonempty groups, it increases by one the number of represented target blocks inside the current coarsening. Starting from one represented block per connected component and ending with at most K target atoms, at most $K - 1$ accepted splits are needed before no atom is impure. □

For finite $\mathcal{U}$ and bounded loss, the event in Theorem 3.7 follows from a standard held-out concentration bound. It suffices to score each accepted split with

$$m = O\left(\frac{1}{\gamma^2} \log \frac{K\,|\mathcal{U}|}{\delta}\right)$$

independent validation samples, up to constants and the bounded-loss range.

3.8 A finite-class oracle inequality

The same finite model gives a structural-risk bound for searching over refinements. Let $\mathfrak{P}_B(\mathcal{U})$ be the set of partitions obtainable from an initial partition Π_0 by at most B distinctions from finite $\mathcal{U}$. If Π_0 has m_0 atoms, every $\Pi \in \mathfrak{P}_B(\mathcal{U})$ has at most $m_0 + B$ atoms, and the number of distinction sequences is at most $|\mathcal{U}|^B$.

Theorem 3.8 (Finite-class refinement oracle inequality)**.** *Assume $L \in [0,1]$ and A is finite. Let $\widehat{R}_L(\Pi)$ be the empirical risk of the best atomwise action for Π on n iid validation samples, and let*

$$\widehat{\Pi} \in \operatorname*{argmin}_{\Pi \in \mathfrak{P}_B(\mathcal{U})} \left\{ \widehat{R}_L(\Pi) + \lambda \, |\Pi| \right\}.$$

Then with probability at least $1 - \delta$,

$$R_L(\widehat{\Pi}) + \lambda \left| \widehat{\Pi} \right| \leq \min_{\Pi \in \mathfrak{P}_B(\mathcal{U})} \{ R_L(\Pi) + \lambda \, |\Pi| \} + 2 \sqrt{\frac{B \log |\mathcal{U}| + (m_0 + B) \log |\mathsf{A}| + \log(2/\delta)}{2n}}.$$

Proof. For any fixed sequence of at most B distinctions and any assignment of actions to the resulting at most $m_0 + B$ atoms, Hoeffding's inequality controls the empirical-population risk deviation. The number of such partition-action pairs is at most $|\mathcal{U}|^B \, |\mathsf{A}|^{m_0+B}$. A union bound gives a uniform deviation ϵ equal to the square-root term without the leading factor 2. On this event, empirical penalized risk and population penalized risk differ by at most ϵ for every candidate. Applying the usual empirical-minimizer argument gives the displayed 2ϵ bound. □

4 DGM: Nodes Repair, Edges Distinguish

The theory so far gives a diagnostic: decide whether an observed error is better explained by Δ_{repair} at fixed information or by Δ_{refine} from missing distinctions. A runtime implementation therefore needs a state representation that can both repair local statistics and add distinctions. DGM is the data-structure realization of the abstract refinable-partition model.

4.1 Why a graph, not a tree?

DGM uses a graph of runtime concepts. A DGM concept is a data-structure representative of a graph-generated state equipped with a local memory; it is not claimed to be a human-interpretable semantic concept. Local memories live at nodes, and distinctions live on edges between concepts that have been confused. The central object is

$$\text{DGM} = \text{graph-generated information structure} + \text{local memory repair}.$$

The graph is not metadata: its edges generate a finite information structure and gate the read path. The partition and generated-sigma-algebra language follows standard finite measurable-space constructions [Bil95, Kal02]. The main-text statements below give the operational theorem forms; full proofs are in Appendix K.

We separate two objects. *Hard-DGM* is the formal algorithm: fixed anchors and accepted binary edges generate a partition; a repair changes only local actions or memories; a refinement adds a binary generator, so the partition is joined with that generator. *Soft-DGM* is the practical relaxation used for sparse neural hidden states: movable centroids retrieve candidates, smooth edge gates approximate hard eligibility, and routing weights mix local memories. The formal guarantees apply to Hard-DGM directly and to Soft-DGM only under the candidate-recall and margin conditions in Theorem 4.5.

4.2 Hard-DGM state and invariants

Let $\phi_\theta : \mathcal{X} \to \mathbb{R}^d$ be an encoder, write $h = \phi_\theta(x)$, and let $\mathsf{H} \subseteq \mathbb{R}^d$ denote the hidden-state space on which the runtime structure is evaluated. A Distinction Graph Machine has runtime state

$$\mathcal{S}_t = \left(\{a_j, c_j, M_j, n_j\}_{j=1}^{N_t}, G_t\right), \qquad G_t = (V_t, E_t), \qquad V_t = \{1, \dots, N_t\}.$$

Each component has a role forced by the refine-or-repair split. A runtime concept needs a stable identity, so $a_j \in \mathbb{R}^d$ is a frozen distinction anchor. Retrieval may adapt as examples arrive, so $c_j \in \mathbb{R}^d$ is a movable centroid used only by the soft implementation. State inside an adequate concept must be repairable, so M_j is a local memory and n_j is a usage count. If a concept is created because it was confused with existing concepts, that conflict becomes an edge in E_t. A new concept is initialized with $a_j = c_j = h$. Subsequent repair may move c_j, but anchors remain fixed once created, so the hard information structure is unchanged by centroid repair.

An edge $e = (i, j)$ stores a linear distinction

$$u_e = \frac{a_i - a_j}{\|a_i - a_j\|_2}, \qquad b_e = \frac{1}{2} u_e^\top (a_i + a_j),$$

with side labels $s_{e,i} = +1$ and $s_{e,j} = -1$. Its binary generator is

$$g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}.$$

The edge set induces an information structure on H. Define the graph signature

$$\sigma_t^G(h) = \big(g_e(h)\big)_{e \in E_t} \in \{0,1\}^{E_t}.$$

Two hidden states are indistinguishable by the current graph if their signatures agree. The nonempty level sets

$$A_a^t = \{h \in \mathsf{H} : \sigma_t^G(h) = a\}, \qquad a \in \{0,1\}^{E_t},$$

form a finite partition $\mathcal{P}_t^G$ of hidden-state space, and the generated sigma-algebra is $\sigma(\mathcal{P}_t^G)$. Adding a new edge is therefore a join with one binary generator:

$$\mathcal{P}_{t+1}^G = \mathcal{P}_t^G \vee \sigma(g_e).$$

Anchors and edges generate the hard distinction skeleton. Centroids and local memories are adaptive statistics attached to states or sparse neighborhoods of this graph-generated partition.

Theorem 4.1 (Anchor distinction separates its endpoints)**.** *If $a_i \neq a_j$ and u_e, b_e are defined as above for edge $e = (i, j)$, then*

$$u_e^\top a_i - b_e = \frac{1}{2}\|a_i - a_j\|_2, \qquad u_e^\top a_j - b_e = -\frac{1}{2}\|a_i - a_j\|_2.$$

Thus each edge distinction separates the two anchors that caused it.

Theorem 4.2 (DGM graph generates a finite information structure)**.** *For any finite edge set E_t, equality of signatures $\sigma_t^G(h)$ is an equivalence relation whose classes form a finite partition $\mathcal{P}_t^G$. Adding one edge g refines the partition by $\mathcal{P}_t^G \vee \sigma(g)$.*

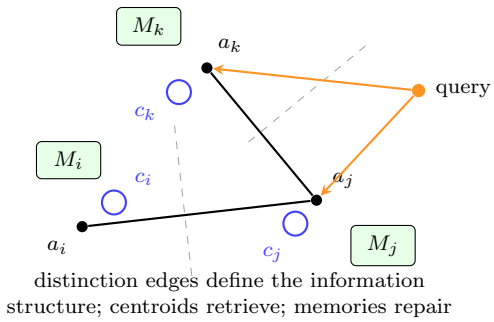

Figure 1: DGM state. Anchors are stable concept identities, centroids are movable retrieval statistics, distinction edges generate the information structure, and concept memories are repaired after a concept is judged adequate.

4.3 Soft-DGM edge-gated prediction

The hard route selects the nearest eligible anchor using fixed anchors and accepted edges, so repairs that change only memories and centroids do not change the hard partition. Given h, Soft-DGM instead computes retrieval distances $d_j(h) = \|h - c_j\|_2^2$ to movable centroids. A centroid-only top-k set is not sufficient for the formal hard-route guarantee: if the hard eligible anchor never enters the candidate set, edge gates cannot recover it. We therefore use a recall-oriented candidate set

$$N(h) = N_{k_c}^{\text{centroid}}(h) \cup N_{k_a}^{\text{anchor}}(h) \cup \text{GraphNeighbors}\Big(N_{k_c}^{\text{centroid}}(h)\Big),$$

with duplicates removed and an optional budget cap after the union. The first term gives adaptive locality, the second protects the anchor-defined hard route, and the third recovers concepts connected by recent consequence conflicts. In exact small-scale implementations one may take $N(h) = \{1, \dots, N_t\}$. If $\text{Inc}_t(j)$ is the set of edges incident to concept j, define the soft compatibility of h with concept j by

$$\log \chi_j(h) = \frac{1}{\sqrt{\max\{1, |\text{Inc}_t(j)|\}}} \sum_{e \in \text{Inc}_t(j)} \beta_e \log \text{sigmoid}\left(\frac{s_{e,j}(u_e^\top h - b_e)}{\tau_e}\right),$$

where structural distinction edges have $\beta_e \in (0, 1]$ and the empty sum gives $\chi_j(h) = 1$. The degree normalization avoids systematically suppressing high-degree concepts while preserving hard rejection in the low-temperature limit. The edge-gated routing weights are

$$\alpha_j(h) = \frac{\exp(-d_j(h)/T)\chi_j(h)}{\sum_{r \in N(h)} \exp(-d_r(h)/T)\chi_r(h)}, \qquad j \in N(h). \tag{10}$$

The prediction is the local mixture

$$\hat{y} = \sum_{j \in N(h)} \alpha_j(h) M_j(h).$$

The hard route makes the information-structure connection explicit. A concept j is eligible for h if every incident edge bit places h on the side recorded for j:

$$\begin{cases} g_e(h) = 1, & s_{e,j} = +1, \\ g_e(h) = 0, & s_{e,j} = -1, \end{cases} \qquad \text{for every } e \in \text{Inc}_t(j).$$

This convention assigns boundary points to the nonpositive side because $g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}$. The exact hard route used in the refinement theory routes to the nearest eligible anchor, falling back to the nearest anchor if no eligible concept is available. The soft read in (10) is the trainable implementation: it may retrieve by movable centroids, but those centroids are retrieval statistics rather than formal information cells.

The distinction partition alone does not determine the whole exact hard route, because eligible concepts still need to be ordered. Anchor distances provide this stable order geometry. Let $\mathcal{F}_t^{\text{edge}} = \sigma(g_e : e \in E_t)$ and let $\mathcal{F}_t^{\text{anchor-order}}$ be the finite sigma-algebra generated by all pairwise anchor-distance comparison events

$$\{h : \|h - a_i\|_2^2 \le \|h - a_j\|_2^2\}, \qquad 1 \le i, j \le N_t,$$

with deterministic tie-breaking. The hard routing skeleton is measurable with respect to

$$\mathcal{F}_t^{\text{DGM}} = \mathcal{F}_t^{\text{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}.$$

Equivalently, the exact DGM information map can be written as

$$I_{\mathcal{S}_t}(h) = ((g_e(h))_{e \in E_t}, \text{HardRoute}_{\mathcal{S}_t}(h)), \qquad \mathcal{F}_t^{\text{DGM}} = \sigma(I_{\mathcal{S}_t}).$$

Thus the exact information-structure theory applies to the anchor-based hard route. The soft read keeps the same graph skeleton but may use movable centroids and smooth gates for locality. It should therefore be read as a differentiable relaxation of the anchor-generated information map, not as a different formal refinement.

Theorem 4.3 (Anchor-based hard routing is graph-plus-anchor-order measurable)**.** *With finite E_t, finite frozen anchors $a_1, \ldots, a_{N_t}$, and deterministic tie-breaking, the exact hard-route selected concept index, defined as the nearest eligible anchor with fallback to the nearest anchor, is measurable with respect to $\mathcal{F}_t^{\text{DGM}} = \mathcal{F}_t^{\text{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}$.*

The next statement is analytic rather than structural: it applies to whichever fixed distances a soft read uses. With anchor distances it recovers the exact hard route; with centroid distances it describes the behavior of the soft retrieval approximation. It also exposes the candidate-recall requirement that links the formal hard route to the sparse implementation.

Condition 4.4 (Candidate recall)**.** *Let $j^\star(h)$ be the unique concept selected by the exact hard route for query h. The sparse soft read satisfies candidate recall at h if*

$$j^\star(h) \in N(h).$$

If this condition fails, the soft read has no guarantee of selecting the hard route, regardless of how sharp the edge gates are.

Theorem 4.5 (Soft routing approaches hard routing)**.** *Assume candidate recall holds for h. If the hard-route selected concept is unique within $N(h)$ with distance margin $\delta > 0$, then its soft routing weight tends to one exponentially as $T \to 0$. If every wrong-side candidate violates some incident structural edge with weight at least $\beta_{\min} > 0$, signed margin at least $\delta_e > 0$, and degree at most $D_{\max}$, then each wrong-side compatibility decays uniformly after degree normalization. If at least one eligible reference candidate has compatibility bounded below and the wrong-side/reference distance-score ratios are bounded, then total wrong-side routing mass vanishes as the edge temperature $\tau_e \to 0$. Without candidate recall, the theorem gives no correctness guarantee for an omitted hard-route concept.*

Algorithm 1 PREDICT for a Distinction Graph Machine

Input: state $\mathcal{S}_t = (\{a_j, c_j, M_j, n_j\}_{j=1}^{N_t}, G_t)$, encoder ϕ_θ, example x
$h \leftarrow \phi_\theta(x)$
$N(h) \leftarrow N_{k_c}^{\text{centroid}}(h) \cup N_{k_a}^{\text{anchor}}(h) \cup \text{GraphNeighbors}(N_{k_c}^{\text{centroid}}(h))$
for $j \in N(h)$ **do**
    compute edge compatibility $\chi_j(h)$ from incident distinctions
    compute edge-gated weight $\alpha_j(h)$ by (10)
end for
return $\sum_{j \in N(h)} \alpha_j(h) M_j(h)$

4.4 Refine or repair on a graph

After observing consequence y, DGM scores empirical versions of the two gaps in Theorem 2.2. A repair candidate keeps the DGM information map fixed and updates only local state. A refinement candidate changes the information map by adding an anchored concept and distinction edges. The committed operation is the one whose held-out gain justifies its cost and safety margin.

Let $j^\star$ be the highest-weight concept under (10). Define local surprise

$$\rho_t = L(y, M_{j^\star}(h)).$$

Define neighborhood consequence disagreement on $N(h)$ by the task-appropriate local prediction divergence. For classification one may use

$$D_t = \max_{i,r \in N(h)} \text{TV}(p_i(\cdot \mid h), p_r(\cdot \mid h)),$$

where p_j is the distribution represented by memory M_j. For regression one may use

$$D_t = \max_{i,r \in N(h)} \|\mu_i(h) - \mu_r(h)\|_2 ,$$

and for language modeling the log-loss surprise $-\log p_{j^\star}(x_{t+1} \mid x_{\leq t})$ is the local disagreement signal. These quantities are proposal signals. They decide which candidates are worth scoring; they do not define repair or refinement.

All gap estimates below are computed on a held-out scoring buffer $\mathcal{B}_t^{\text{score}}$. Refinement proposals may use a disjoint proposal buffer $\mathcal{B}_t^{\text{prop}}$ together with the current observation; the scoring buffer is held out from proposal generation. A practical implementation can maintain the split by reservoir sampling incoming observations into the two buffers. If an implementation reuses one buffer for both proposal and scoring, the empirical rule still defines a heuristic, but Theorem 4.8 applies to the sample-split version.

The edges created by a refinement are attached to a concrete confused-neighbor set. Let $\text{Div}_{\mathcal{Y}}(M_i, M_r; h)$ denote a task-appropriate consequence divergence, such as total variation for classifiers, Euclidean mean distance for regression memories, or next-token distribution divergence for language models. Define

$$\text{Confused}(h, y) = \{i \in N(h) : L(y, M_i(h)) > \tau_{\text{conflict}} \text{ or } \text{Div}_{\mathcal{Y}}(M_i, M_{j^\star}; h) > \tau_{\text{disagree}}\} .$$

When a budget of at most q new edges is used, DGM takes $\text{Confused}_q(h, y)$ to be the q highest-scoring members of this set under

$$\text{score}_i(h, y) = \alpha_i(h) L(y, M_i(h)) + \gamma_{\text{div}} \text{Div}_{\mathcal{Y}}(M_i, M_{j^\star}; h).$$

If the set is empty but refinement is otherwise accepted, the implementation falls back to the highest-score candidates in $N(h) \setminus \{j^\star\}$. Thus "confused neighbors" denotes a concrete operator rather than an informal phrase.

Repair is guarded rather than blindly committed. The implementation proposes a small set of repair candidates, such as a memory-only update, a memory-plus-centroid update, and one or more reduced centroid steps:

$$\mathcal{C}_{\text{repair}}(\mathcal{S}_t, h, y) = \{\mathcal{S}_t^{\text{mem}}, \mathcal{S}_t^{\text{mem+centroid}}, \mathcal{S}_t^{\text{shrink}}, \mathcal{S}_t\}.$$

Each candidate keeps anchors and edges fixed, so it preserves $I_{\mathcal{S}_t}$ and $\mathcal{F}_t^{\text{DGM}}$. On the scoring buffer, the empirical repair gain is

$$\widehat{\Delta}_{\text{repair}} = \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \min_{\mathcal{S}' \in \mathcal{C}_{\text{repair}}} \widehat{R}(\mathcal{S}'; \mathcal{B}_t^{\text{score}}). \tag{11}$$

DGM's guarded repair candidate is

$$\mathcal{S}_{t+1} \in \operatorname*{argmin}_{\mathcal{S}' \in \mathcal{C}_{\text{repair}}} \widehat{R}(\mathcal{S}'; \mathcal{B}_t^{\text{score}}) \quad \text{subject to} \quad \widehat{R}(\mathcal{S}'; \mathcal{B}_t^{\text{score}}) \le \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}). \tag{12}$$

The identity state is in $\mathcal{C}_{\text{repair}}$, so $\widehat{\Delta}_{\text{repair}} \ge 0$. Thus monotone repair is an enforced algorithmic property, not merely a theorem assumption. The anchor $a_{j^\star}$ is never moved by repair. Consequently, local memory repair and centroid adaptation do not rewrite previously accepted edge generators.

A candidate creates

$$a_{\text{new}} = h, \qquad c_{\text{new}} = h, \qquad M_{\text{new}} = \text{InitMemory}(h, y),$$

and adds distinction edges from the new anchored concept to $\text{Confused}_q(h, y)$. Let $\widehat{R}(\mathcal{S}; \mathcal{B}_t^{\text{score}})$ be the empirical local loss on the scoring buffer after routing with state $\mathcal{S}$. The empirical penalized refinement gain is

$$\widehat{\Delta}_{\text{refine}} = \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \widehat{R}(\mathcal{S}_t^{\text{new}}; \mathcal{B}_t^{\text{score}}) - \lambda_{\text{concept}} - \lambda_{\text{edge}} q. \tag{13}$$

The candidate is accepted only if it beats the repair gain by a safety margin:

$$\widehat{\Delta}_{\text{refine}} > \max\{\widehat{\Delta}_{\text{repair}}, 0\} + \tau + 2\epsilon_m \qquad \text{and} \qquad D_t > \tau_{\text{disagree}}. \tag{14}$$

Here ϵ_m is a statistical safety margin for a scoring buffer of size $m = |\mathcal{B}_t^{\text{score}}|$; it can be set to zero for a purely empirical implementation, or to the adaptive validation margin in Theorem 4.8. If the refinement test fails, DGM commits the guarded repair candidate in (12). This makes refinement a statistically guarded gap comparison rather than an unscored growth heuristic.

Algorithm 2 OBSERVE for a Distinction Graph Machine

Input: state $\mathcal{S}_t$, labeled consequence (x, y)
$h \leftarrow \phi_\theta(x)$
if $N_t = 0$ **then**
  create anchor and centroid $a_1 = c_1 = h$ with memory $\text{InitMemory}(h, y)$
  return updated state
end if
retrieve $N(h)$ and compute edge-gated routing weights
$j^\star \leftarrow \text{argmax}_{j \in N(h)} \alpha_j(h)$
$\rho \leftarrow L(y, M_{j^\star}(h))$
compute neighborhood disagreement D_t
propose repair candidates with anchors and edges fixed
compute empirical repair gain $\widehat{\Delta}_{\text{repair}}$ by (11)
if $\rho > \tau_{\text{repair}}$ or $D_t > \tau_{\text{disagree}}$ **then**
  form a candidate new anchor/centroid from h and $\mathcal{B}_t^{\text{prop}}$, with candidate edges to $\text{Confused}_q(h, y)$
  estimate penalized refinement gain $\widehat{\Delta}_{\text{refine}}$ on $\mathcal{B}_t^{\text{score}}$ by (13)
  if $\widehat{\Delta}_{\text{refine}} > \max\{\widehat{\Delta}_{\text{repair}}, 0\} + \tau + 2\epsilon_m$ and $D_t > \tau_{\text{disagree}}$ **then**
    commit the new anchored concept and its distinction edges
    return updated state
  end if
end if
commit the best candidate satisfying the guarded repair test (12)
return updated state

Definition 4.6 (Guarded monotone local repair)**.** *A guarded repair operator is monotone on scoring buffer $\mathcal{B}_t^{\text{score}}$ if it commits a candidate state $\mathcal{S}_{t+1}$ only when local memories and retrieval centroids may change, anchors and edges remain fixed, and*

$$\widehat{R}(\mathcal{S}_{t+1}; \mathcal{B}_t^{\text{score}}) \leq \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}).$$

The identity candidate $\mathcal{S}_{t+1} = \mathcal{S}_t$ is always allowed, so the guard is feasible even when every nontrivial repair would hurt the scoring buffer.

Theorem 4.7 (Refine-or-repair objective descent)**.** *Let the local scoring objective on a fixed scoring buffer $\mathcal{B}_t^{\text{score}}$ be*

$$J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}) = \widehat{R}(\mathcal{S}; \mathcal{B}_t^{\text{score}}) + \lambda_{\text{concept}} N_{\text{concept}}(\mathcal{S}) + \lambda_{\text{edge}} |E(\mathcal{S})|.$$

If repair is committed by the guarded rule (12), then $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. If a refinement adding one anchored concept and q edges is accepted only under (14), then the empirical penalized objective $J_{\mathcal{B}_t^{\text{score}}}$ decreases by more than $\widehat{\Delta}_{\text{repair}} + \tau + 2\epsilon_m$ and in particular by more than $\tau + 2\epsilon_m$.

This is a local-buffer statement. It does not imply monotone descent of a global objective when the scoring buffer changes over time; a global finiteness claim requires a common potential or validation objective.

Theorem 4.8 (Statistically safe adaptive refinement)**.** *Assume losses lie in $[0, 1]$. Let $(\mathfrak{H}_t)$ be the filtration containing the past stream, the current DGM state, and all proposal-buffer information*

available before scoring at time t. Let $\mathcal{C}_t$ be a finite candidate class that is predictable with respect to $\mathfrak{H}_t$, and let $\mathcal{B}_t^{\text{score}}$ contain m_t samples independent of $\mathfrak{H}_t$ conditional on the current scoring distribution. Define

$$\epsilon_t = \sqrt{\frac{\log(4t^2(|\mathcal{C}_t|+1)/\delta)}{2m_t}}.$$

Then, with probability at least $1-\delta$, for every time t and every $g \in \mathcal{C}_t$,

$$\left|\widehat{\Delta}_t(g) - \Delta_t(g)\right| \le 2\epsilon_t.$$

Hence an accepted refinement satisfying

$$\widehat{\Delta}_t(g) > \lambda \operatorname{cost}(g) + \tau + 2\epsilon_t$$

has positive population gain after paying complexity cost and margin τ, uniformly over all accepted times.

Corollary 4.9 (Bounded language-model refinement score)**.** *For language modeling, raw next-token negative log likelihood is unbounded. Fix $L_{\max} > 0$ and define the clipped normalized score*

$$\ell_{\text{clip}}(x_{\le t}, x_{t+1}) = \frac{1}{L_{\max}} \min\{-\log p(x_{t+1} \mid x_{\le t}), L_{\max}\}.$$

If refinement proposal and scoring use ℓ_{clip}, then Theorem 4.8 applies to clipped-loss refinement gain. An accepted refinement has positive clipped population gain after cost and margin with the stated high probability. This corollary does not claim positive gain for the raw unbounded negative log likelihood without additional tail assumptions.

Theorem 4.10 (Anchor-stable DGM information structure)**.** *Let*

$$\mathcal{F}_t^{\text{DGM}} = \sigma(g_e : e \in E_t) \vee \sigma\Big(\{h : \|h-a_i\|_2^2 \le \|h-a_j\|_2^2\} : 1 \le i,j \le N_t\Big).$$

If a repair step changes only retrieval centroids c_j, local memories M_j, and counts n_j, while keeping anchors a_j and edges E_t fixed, then $\mathcal{F}_{t+1}^{\text{DGM}} = \mathcal{F}_t^{\text{DGM}}$. If a refinement adds a new anchor and new edge generators, then the updated information structure is the join of $\mathcal{F}_t^{\text{DGM}}$ with the new anchor-order events and the new edge generators.

Corollary 4.11 (Finite number of accepted refinements)**.** *If there is a common nonnegative potential $\Phi(\mathcal{S})$ and every accepted refinement decreases Φ by at least $\tau > 0$, then the number of accepted refinements is at most $\Phi(\mathcal{S}_0)/\tau$. In particular, this applies to a fixed scoring buffer or validation objective. The local-buffer descent theorem alone should not be read as a global finiteness result when the scoring buffer changes over time.*

4.5 Data-structure operations

Viewed as a dynamic partition data structure, DGM implements four operations:

$$\text{Query}(h), \qquad \text{Repair}(h,y), \qquad \text{Refine}(h,y,g), \qquad \text{Score}(g,\mathcal{B}).$$

The maintained invariant is

$$\begin{aligned} \text{Repair} \quad &\Longrightarrow \quad \mathcal{P}^G(\mathcal{S}_{t+1}) = \mathcal{P}^G(\mathcal{S}_t), \\ \text{Refine}(g) \quad &\Longrightarrow \quad \mathcal{P}^G(\mathcal{S}_{t+1}) = \mathcal{P}^G(\mathcal{S}_t) \vee \sigma(g), \end{aligned}$$

where $\mathcal{P}^G(\mathcal{S}_t)$ is the edge-generated hard partition. Thus repair modifies payloads on the current structure, while refinement changes the structure.

Theorem 4.12 (DGM data-structure cost)**.** *Assume N_t concepts in $\mathbb{R}^d$, candidate set size $|N(h)| = k$, selected incident edge count $Q(h)$, retrieval cost $C_{\mathrm{ret}}(N_t, d)$, and selected local-memory evaluation cost C_{mem}. Then*

$$\mathrm{Query}(h) = O(C_{\mathrm{ret}}(N_t, d) + Q(h) + kd + C_{\mathrm{mem}}).$$

A repair adds the cost of scoring the proposed local-memory candidates on the scoring buffer. A refinement creating one anchor-centroid pair and q linear distinction edges adds $O(d + qd)$ state-update time. With sparse categorical memories of average support $\bar{s}$, the state uses

$$O(N_t(2d + \bar{s}) + |E_t|\, d)$$

memory, up to indexing overhead.

The theorem is an implementation bound rather than a statistical guarantee. It says that DGM realizes the abstract operation $\Pi \mapsto \Pi \vee U$ with local graph updates and sparse routing, provided the candidate retrieval layer supplies the relevant hard-route concepts.

4.6 Canonical local memory

The local memory M_j is the repair mechanism inside an already chosen concept. For classification and language modeling, the main-text canonical memory is a calibrated categorical memory rather than an unconstrained embedding score. Concept j stores token or label counts $n_{j,y}$ and $n_j = \sum_y n_{j,y}$. Given a backbone distribution $p_0(y \mid h)$ and smoothing strength $\alpha_0 > 0$, define

$$p_j(y \mid h) = \frac{n_{j,y} + \alpha_0 p_0(y \mid h)}{n_j + \alpha_0}. \tag{15}$$

The routed memory distribution is

$$p_{\mathrm{mem}}(y \mid h) = \sum_{j \in N(h)} \alpha_j(h) p_j(y \mid h), \tag{16}$$

and the final prediction is the convex mixture

$$p_{\mathrm{DGM}}(y \mid h) = (1 - \lambda_t) p_0(y \mid h) + \lambda_t p_{\mathrm{mem}}(y \mid h), \qquad \lambda_t \in [0, 1]. \tag{17}$$

This keeps the read path a valid probability distribution. It also gives the simple safety bound

$$-\log p_{\mathrm{DGM}}(y \mid h) \le -\log p_0(y \mid h) - \log(1 - \lambda_t)$$

whenever $\lambda_t < 1$. In experiments where log loss is central, λ_t should be selected online by a validation rule or by Hedge over a small grid of trust weights.

Repair is the sufficient-statistic update. For every selected concept $j \in N(h)$,

$$n^+_{j,y} = n_{j,y} + \alpha_j(h), \qquad n^+_j = n_j + \alpha_j(h),$$

with all unselected counts unchanged. The low-cost hard-update variant is recovered by setting $\alpha_{j^\star} = 1$ for the highest-weight concept and $\alpha_j = 0$ for all $j \neq j^\star$. Embedding-mean memories and ridge or projection-style local repair are useful implementation variants, but they should be evaluated with calibration metrics before being used for log-loss claims.

4.7 From DGM to DGM-Embedding

The generic DGM object above can be attached to many local memories. For sequence models, Appendix D gives DGM-Embedding as an architectural extension with a read-before-observe protocol and forward-only detached writes.

5 Guarantees

The guarantees are deliberately scoped. They do not assert global optimality of DGM, nor do they assert that a deep backbone can always discover the right representation. They formalize the paper's decision rule: repair-only learners are limited by the Bayes risk of their current partition; useful refinements change the Bayes action; statistically guarded refinement accepts only distinctions whose held-out consequence value clears complexity and concentration margins; and DGM implements this operation as a dynamic graph data structure. Full proof details are in Appendices I, J, and K.

1. Error decomposes into two gaps. Theorem 2.2 states that for current predictor f_t, current partition Π_t, and candidate refined partition $\Pi^\star$,

$$R_L(f_t) - R_L(\Pi^\star) = \Delta_{\text{repair}}(f_t, \Pi_t) + \Delta_{\text{refine}}(\Pi_t, \Pi^\star).$$

The first term is the best possible gain from repairing the predictor at fixed information. The second is the value of changing the information structure.

2. Repair-only learners have linear regret. Theorem 2.3 states that any learner whose predictions remain measurable with respect to a fixed partition Π has expected regret at least

$$T\left(R_L(\Pi) - R_L(\Pi^\star)\right)$$

against a refined oracle $\Pi^\star$. Thus a positive refinement gap is not just a pointwise contradiction; it is a sequential lower bound.

3. Useful refinements change the Bayes action. Theorem 2.6 states that, under uniqueness of the refined Bayes action, a refinement has zero value exactly when the refined action is already measurable under the old information. For finite Y under log loss, Theorem 3.2 sharpens this to

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = \mathbb{E}\, D_{\mathrm{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{G}) \| \mathbb{P}(Y \in \cdot \mid \mathcal{F}))\,.$$

Thus stochastic conflict, not only hard-label contradiction, is enough to make refinement valuable.

4. Missing distinctions obstruct repair. Corollary 2.5 says that if two states have the same information-map value but require different consequences, no predictor measurable with respect to the current information can satisfy both. This is the main theoretical point of the paper:

$$\boxed{\text{if the target separates states that the information map identifies, repair is impossible.}}$$

The result is architecture-independent; it applies to caches, adapters, local memories, and test-time optimizers whenever their update leaves the relevant information structure fixed.

5. Refinement has a canonical minimal form. Theorems 2.7 and 2.8 identify the least information update that makes new evidence observable. In the finite case, a binary distinction splits exactly the atoms it crosses. In the sigma-algebra case, the update is

$$\mathcal{F}_{t+1} = \mathcal{F}_t \vee \sigma(U_t).$$

Section 3 gives the standard value identities that explain when such a distinction is useful: conditional mutual information under log loss and conditional variance reduction under squared loss. Theorem 3.5 further identifies the smallest refinement needed to express the refined Bayes action.

6. Greedy refinement has finite-class guarantees. Theorem 3.7 gives a realizable recovery guarantee: if every impure target atom admits a target-respecting distinction with value at least γ, and split values are estimated accurately enough, greedy refinement recovers the target partition after at most $K-1$ accepted splits. Theorem 3.8 gives a finite-class oracle inequality over partitions reachable by at most B distinctions.

7. Guarded DGM updates descend locally. Theorem 4.7 states the algorithmic guarantee. A guarded repair does not increase the scoring-buffer objective because anchors and edges remain fixed and only a non-worsening local-state candidate is committed. An accepted refinement adds one concept and q edges only when its held-out penalized refinement gain beats the repair gain plus a margin and statistical safety term. Thus DGM is not an unscored growth heuristic: it is a local refine-or-repair decision with an explicit gap comparison.

These statements also define the boundary of the claim. The scoring buffer is local and changes over time, so the descent result is not a global training theorem. Approximate retrieval must satisfy candidate recall for Soft-DGM edge gates to recover the formal Hard-DGM route. Budgeting, merging, and eviction are systems approximations that may discard distinctions and must be evaluated separately.

6 Separating Examples and Sanity Checks

The experiments are separating examples for the formal model, not a broad benchmark suite. Each task contains a repair component, where local memory updates help, and a refinement component, where repair-only learners collapse incompatible consequences into one partition atom. All learners use a prequential protocol: predict first, observe the consequence, then update. The current experiments do not establish language-model-scale competitiveness, and the hidden-regime log-loss results expose the need for the calibrated memory in (17). Implementation details and additional sanity checks are in Appendix F.

6.1 Hidden-regime sequence prediction

The most direct check is a hidden-regime online sequence task. A hidden regime chooses the next-token transition map. The learner is not given the regime label; it observes only a noisy continuous cue and the current token, predicts the next token, and then receives the next-token consequence. The same current token can therefore require different next tokens in different regimes. A repair-only memory averages those consequences, while a refine-and-repair learner can create separate runtime concepts.

Held-out accuracies for the same runs were 0.285 ± 0.049 for cache LM, 0.346 ± 0.106 for budgeted kNN-LM, 0.235 ± 0.024 for nearest-centroid memory, 0.055 ± 0.015 for repair-only DGM,

Model	Runtime state	Preq. acc.	Preq. log loss	Concepts
Cache LM	token counts	0.362 ± 0.029	1.443 ± 0.069	–
Budgeted kNN-LM	64 examples	0.583 ± 0.010	4.342 ± 0.338	64 examples
Nearest-centroid memory	16 centroids	0.278 ± 0.035	2.070 ± 0.112	16
DGM repair only	one local memory	0.079 ± 0.017	2.798 ± 0.006	1
DGM refine+repair	graph memory	$\mathbf{0.617 \pm 0.040}$	5.348 ± 0.625	87.6
Backprop MLP, online	one backprop step	0.292 ± 0.020	2.217 ± 0.054	–

Table 1: Hidden-regime online sequence prediction, averaged over five seeds. The useful operation is to create distinctions when identical tokens have regime-dependent consequences. The refine+repair run creates runtime concepts and had monotone local scoring-objective decrease at accepted refinements in all runs, which improves accuracy. Its poor log loss shows that the uncalibrated memory used in this diagnostic is not adequate for probabilistic claims.

Model	Accepted splits	Prequential accuracy	Test accuracy
Repair only	0	0.491	0.501
Refine+repair	9	0.939	0.967

Table 2: Single-run hard-routing ablation on XOR Gaussian clusters. Local repair alone cannot solve a missing-distinction task, while accepted refinements decrease the local penalized objective and recover the required partition.

0.680 ± 0.065 for refine+repair DGM, and 0.282 ± 0.086 for online backpropagation. The important comparison is structural: repair-only DGM fails because the task is constructed so that one local memory is the wrong information structure. The log-loss column is equally important: a method that wins accuracy while losing NLL is not a calibrated probabilistic runtime learner, so the main algorithm specifies smoothed categorical memories and a backbone mixture before making log-loss claims.

6.2 Repair-only ablation

We also run a hard-routing special case on XOR Gaussian clusters. The repair-only model has one leaf and cannot represent XOR, while refine+repair creates accepted distinctions with objective decrease. This is the smallest test of the obstruction theorem: if incompatible labels lie inside one information cell, local repair has no representation in which to store the answer.

6.3 Controlled contextual XOR

As a graph-memory check, we use a two-context XOR stream. In the first context the label is XOR of two Gaussian cluster signs; in the second context the labels are flipped. The context coordinate is observable, but the learner must discover how it changes the consequence. The offline MLP is a batch upper bound; the online MLP predicts each example and then performs one backpropagation step; DGM uses the same online protocol but may refine the concept graph before repairing local state.

Model	Runtime update	Prequential accuracy	Test accuracy
Backprop MLP, offline	batch backprop	–	0.993 ± 0.002
Backprop MLP, online	one backprop step	0.569 ± 0.065	0.654 ± 0.038
DGM graph memory, online	refine-or-repair	0.804 ± 0.025	0.944 ± 0.026

Table 3: DGM graph memory versus backpropagation on a limited online contextual-XOR stream. Offline backprop is a batch upper bound. The relevant runtime comparison is between online backpropagation and online distinction refinement. The DGM run has monotone local scoring-objective decrease at accepted refinements in all runs.

Task	Offline backprop	Online backprop	Online DGM memory
Synthetic LM	0.931 ± 0.071	0.125 ± 0.021	0.912 ± 0.077

Table 4: Synthetic next-token stream. The task isolates runtime adaptation: the hidden context determines the token transition rule, so the useful operation is to remember a local prefix-to-token consequence after observing it. DGM memory approaches the offline batch MLP while one-step online backpropagation adapts slowly.

6.4 Synthetic next-token stream

Finally, we include a synthetic language-modeling stream. Each document context induces a different next-token transition map. The learner observes a prefix representation and must predict the next token before seeing it. After the next token is revealed, DGM memory stores the prefix type and next-token consequence; online backpropagation again receives one gradient step per token. This is a controlled next-token diagnostic, not evidence that DGM-Embedding is a competitive cache or kNN-LM replacement on real corpora.

These experiments do not claim that DGM is a finished competitive language-modeling system. They check the mechanism predicted by the theory: when error is caused by a missing distinction, methods that only repair state are forced to mix incompatible consequences; a refine-and-repair learner can create the missing runtime concept and then repair it locally.

7 Related Work

The relevant comparison is not whether DGM uses centers, memory, routing, or online updates. All of these ingredients have long histories. The comparison is whether a runtime learner separates two operations that are often collapsed: repairing state inside the current representation and refining the representation when consequences reveal a missing distinction.

Repair without refinement. Cache language models and kNN-LMs store or retrieve context-token examples at inference time [GCJ17, KLJ+20]. Fast weights, linear attention memories, passive-aggressive updates, online ridge/RLS, and projection-style memories repair associative state inside a fixed geometry [BHM+16, KVPF20, SIS21, CDK+06, Vov97, Bre67, TA77, HK70]. Test-time adaptation and test-time training update model state or parameters by optimizing an inference-time surrogate [SWL+20, WSL+21, NWZ+22, ZLF22, SLD+25]. These methods address the Δ_{repair} side of runtime learning; DGM asks the prior structural question of whether the error is repairable under the current distinctions.

Refinement without local repair. Decision trees, online trees, Hoeffding trees, adaptive trees, and Mondrian forests refine leaves using information gain, variance reduction, or streaming split tests [BFOS84, DH00, BG09, MWS18, LRT14]. Adaptive Resonance Theory and prototype methods create or move categories under mismatch [CG87b, CG87a, Gro13, Koh90, Fri95, SSZ17]. DGM shares the refinement motivation, but stores distinctions as local graph edges between runtime concepts and attaches repairable memories to the concepts after they are created.

Routing without runtime concept creation. Mixture-of-experts, Switch Transformers, and Product-Key Memory route queries to sparse trained capacity [SMM+17, FZS22, LSR+19]. Memory Networks, Neural Turing Machines, and Differentiable Neural Computers learn external memory controllers [WCB15, SWF+15, GWD14, GWR+16]. DGM differs in write semantics: the routed objects are runtime-created concepts whose edges generate an information structure and whose memories are repaired locally.

Appendix C gives the extended related-work discussion. The short version is that DGM combines refinement and repair while keeping their roles separate:

$$\Delta_{\text{refine}} \text{ dominates} \Rightarrow \text{refine}, \qquad \Delta_{\text{repair}} \text{ dominates} \Rightarrow \text{repair}.$$

8 Conclusion

Runtime learning fails when it repairs errors that actually require new distinctions. In the refinable-partition model, this failure is a quotient error: a repair-only learner remains trapped inside the equivalence relation induced by its current state and suffers linear regret against a refined oracle whenever the refinement gap is positive. The sharper characterization is loss-relevant: a refinement matters when it changes the Bayes action, or under log loss when it changes the conditional outcome distribution. Distinction Graph Machines turn this separation into an explicit runtime learner with anchored concepts, calibrated local memories, distinction edges, and a read-before-observe update rule.

The theory explains why this separation matters. Contradictions inside an information cell cannot be solved by optimizing a predictor that still factors through the same information map. They require refinement. Once the local concept is adequate, its state can be repaired by sufficient-statistic, ridge, or projection-memory updates. DGM implements both operations in one state: concepts hold local memories, and distinction edges generate the information structure that gates the read path.

The resulting picture places gradient learning, local repair, and distinction refinement in one hierarchy. Backpropagation moves slow parameters; local memory repairs fast state; DGM adds the operation of changing the information structure on which those states are defined. The remaining empirical burden is substantial: the mechanism checks here should be extended to calibrated NLL, stronger online baselines, memory-latency budgets, and realistic frozen-model adaptation streams before making large-scale language-model claims.

References

[Ber85] James O. Berger. *Statistical Decision Theory and Bayesian Analysis*. Springer, 2 edition, 1985.

[BFOS84] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

[BG09] Albert Bifet and Ricard Gavaldà. Adaptive learning from evolving data streams. In *Advances in Intelligent Data Analysis VIII*, volume 5772 of *Lecture Notes in Computer Science*, pages 249–260. Springer, 2009.

[BHM+16] Jimmy Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

[Bil95] Patrick Billingsley. *Probability and Measure*. Wiley, 3 edition, 1995.

[Bla53] David Blackwell. Equivalent comparisons of experiments. *The Annals of Mathematical Statistics*, 24(2):265–272, 1953.

[BMH+22] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 2206–2240, 2022.

[Bre67] Lev M. Bregman. The relaxation method of finding the common point of convex sets and its application to the solution of problems in convex programming. *USSR Computational Mathematics and Mathematical Physics*, 7(3):200–217, 1967.

[CDK+06] Koby Crammer, Ofer Dekel, Joseph Keshet, Shai Shalev-Shwartz, and Yoram Singer. Online passive-aggressive algorithms. *Journal of Machine Learning Research*, 7:551–585, 2006.

[CG87a] Gail A. Carpenter and Stephen Grossberg. ART 2: Self-organization of stable category recognition codes for analog input patterns. *Applied Optics*, 26(23):4919–4930, 1987.

[CG87b] Gail A. Carpenter and Stephen Grossberg. A massively parallel architecture for a self-organizing neural pattern recognition machine. *Computer Vision, Graphics, and Image Processing*, 37(1):54–115, 1987.

[CT06] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley, 2 edition, 2006.

[DGL96] Luc Devroye, László Györfi, and Gábor Lugosi. *A Probabilistic Theory of Pattern Recognition*. Springer, 1996.

[DH00] Pedro Domingos and Geoff Hulten. Mining high-speed data streams. In *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 71–80, 2000.

[Fri95] Bernd Fritzke. A growing neural gas network learns topologies. In *Advances in Neural Information Processing Systems*, volume 7, pages 625–632, 1995.

[FZS22] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[GCJ17] Edouard Grave, Moustapha M. Cisse, and Armand Joulin. Improving neural language models with a continuous cache. In *International Conference on Learning Representations*, 2017.

[GR07] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

[Gro13] Stephen Grossberg. Adaptive resonance theory: How a brain learns to consciously attend, learn, and recognize a changing world. *Neural Networks*, 37:1–47, 2013.

[GW99] Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, 1999.

[GWD14] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines, 2014.

[GWR+16] Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, 2016.

[HK70] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[Hoe63] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

[IM98] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, pages 604–613, 1998.

[JDJ19] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[Kal02] Olav Kallenberg. *Foundations of Modern Probability*. Springer, 2 edition, 2002.

[KLJ+20] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020.

[KNT+20] Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim, and Percy Liang. Concept bottleneck models. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5338–5348, 2020.

[Koh90] Teuvo Kohonen. The self-organizing map. *Proceedings of the IEEE*, 78(9):1464–1480, 1990.

[Koh95] Teuvo Kohonen. *Self-Organizing Maps*, volume 30 of *Springer Series in Information Sciences*. Springer, 1995.

[KPR+17] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[KVPF20] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165, 2020.

[LPP+20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[LPR17] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[LRT14] Balaji Lakshminarayanan, Daniel M. Roy, and Yee Whye Teh. Mondrian forests: Efficient online random forests. In *Advances in Neural Information Processing Systems*, volume 27, 2014.

[LSR+19] Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Herve Jegou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[MS91] Thomas M. Martinetz and Klaus J. Schulten. A neural-gas network learns topologies. In *Artificial Neural Networks*, pages 397–402, 1991.

[MWS18] Chaitanya Manapragada, Geoffrey I. Webb, and Mahsa Salehi. Extremely fast decision tree. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1953–1962, 2018.

[NWZ+22] Shuaicheng Niu, Jiaxiang Wu, Yifan Zhang, Yaofo Chen, Shihua Zheng, Peilin Zhao, and Mingkui Tan. Efficient test-time model adaptation without forgetting. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 16888–16905, 2022.

[Pen55] Roger Penrose. A generalized inverse for matrices. *Mathematical Proceedings of the Cambridge Philosophical Society*, 51(3):406–413, 1955.

[RHH+16] Jack W. Rae, Jonathan J. Hunt, Tim Harley, Ivo Danihelka, Andrew Senior, Greg Wayne, Alex Graves, and Timothy Lillicrap. Scaling memory-augmented neural networks with sparse reads and writes. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

[RHW86] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[RRD+16] Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks, 2016.

[SIS21] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9355–9366, 2021.

[SLD+25] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57503–57522, 2025.

[SMM+17] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

[SSZ17] Jake Snell, Kevin Swersky, and Richard S. Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[SWF+15] Sainbayar Sukhbaatar, Jason Weston, Rob Fergus, et al. End-to-end memory networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[SWL+20] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9229–9248, 2020.

[TA77] Andrey N. Tikhonov and Vasiliy Y. Arsenin. *Solutions of Ill-Posed Problems*. Winston, 1977.

[Vov97] Volodya Vovk. Competitive on-line linear regression. In *Advances in Neural Information Processing Systems*, volume 10, 1997.

[WCB15] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *International Conference on Learning Representations*, 2015.

[WRHS22] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

[WSL+21] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021.

[YYLH18] Jaehong Yoon, Eunho Yang, Jeongtae Lee, and Sung Ju Hwang. Lifelong learning with dynamically expandable networks. In *International Conference on Learning Representations*, 2018.

[ZLF22] Marvin Zhang, Sergey Levine, and Chelsea Finn. MEMO: Test time robustness via adaptation and augmentation. In *Advances in Neural Information Processing Systems*, volume 35, pages 38629–38642, 2022.

Appendix

A Appendix Roadmap

The appendix keeps secondary material out of the main method story. It first states limitations, then gives extended related work, the DGM-Embedding architectural extension, implementation extensions, additional computational checks, proof sketches, the projection-memory bridge, and formal proofs.

- **Part 1.** Appendix B clarifies what the theory proves and what it leaves to modeling, statistics, and experiments.
- **Part 2.** Appendix C gives the extended related-work discussion.
- **Part 3.** Appendix D gives DGM-Embedding as an architectural extension; Appendix E collects tree special cases, data-structure costs, the optional adapter variant, systems notes, and closed-form local repair.
- **Part 4.** Appendix F reports secondary computational checks.
- **Part 5.** Appendix G gives proof sketches, and Appendix H gives the Projection Memory and CPM bridge.
- **Part 6.** Appendices I, J, and K give the formal proofs.

B Limitations

The theory identifies when a distinction is necessary or valuable at the population-information level. It does not solve the search problem of finding the best distinction in a rich hypothesis class. The online algorithm is greedy and local: its refinement rule concerns local concepts and their nearby conflicts, not global optimality over all information structures.

The risk identities assume correctly estimated conditional distributions or conditional means. With finite data, refinement can increase estimation error, so useful population distinctions may be statistically unsafe. Complexity penalties, thresholds, validation, Bayesian priors, or minimum leaf sizes are needed in practical systems.

The DGM method specifies one canonical refinement mechanism, based on concepts and local distinction edges, but this is not claimed to be optimal. Rich domains may require learned concept-creation rules, approximate nearest-neighbor indices, nonlinear distinction edges, and better merge or forgetting policies than the default budget notes in Appendix E. The bridge to Projection Memory and CPM is exact for one-hot finite partition memories. Continuous neural representations need additional modeling choices: when a continuous conflict should create a new concept, how local projection memories should be inherited, and how refined information structures interact with differentiable backbones.

Soft sparse retrieval introduces an additional recall condition. Edge gates can reject incompatible candidates, but they cannot select a hard-compatible concept that was never retrieved. The union candidate set in Section 4 reduces this risk, but approximate retrieval remains an engineering assumption that should be measured in large systems.

The large-model layer is an architectural path rather than a completed scaling theorem. It gives a trainable read path, a detached write path, and a default budget policy with explicit

complexity, but it does not prove optimization guarantees for deep nonconvex backbones, nor does it solve distributed systems issues such as load balancing or stale concept shards. Budget maintenance is also outside the formal refinement theorem: merge, eviction, and compression may discard distinctions, so their effect must be evaluated as a resource-management approximation.

The experiments are computational checks, not a benchmark suite. A complete empirical evaluation should include stronger online and nonparametric baselines, ablations of repair-only, refine-only, edge-free, and edge-gated variants, and tasks where hidden context changes force new distinctions.

C Related Work

The relevant comparison is not whether DGM uses centers, memory, routing, or online updates. All of these ingredients have long histories. The question is whether a runtime learner distinguishes two operations that are often collapsed: repairing state inside the current representation and refining the representation when consequences reveal a missing distinction.

C.1 Methods that repair without refining

Cache language models and kNN-LMs store or retrieve context-token examples at inference time [GCJ17, KLJ+20]. Memorizing Transformers, RETRO, RAG, and sparse retrieval systems extend this idea with larger or external stores [WRHS22, BMH+22, LPP+20, RHH+16]. These methods make past examples available, but the default response to surprise is more storage or retrieval, not the creation of a new runtime concept forced by consequence conflict.

Fast weights, linear attention memories, delta-rule updates, passive-aggressive learning, online ridge/RLS, and Projection Memory repair associative state inside a fixed geometry [BHM+16, KVPF20, SIS21, CDK+06, Vov97, Bre67, TA77, HK70]. Conservative Projection Memory (CPM) is especially close to our repair side because it detects zero innovation with nonzero residual as a conflict. DGM uses this diagnosis differently: a zero-innovation conflict is evidence that the current information cell is too coarse, so projection-style repair should stop until the information structure is refined.

Test-time adaptation and test-time training update model state or parameters by optimizing an inference-time objective [SWL+20, WSL+21, NWZ+22, ZLF22, SLD+25]. These methods ask what surrogate loss should be optimized at test time. DGM asks a prior structural question: is the error repairable under the current distinctions, or does it reveal a missing distinction?

These methods solve the repair-gap side of runtime learning. They do not, by themselves, provide an explicit consequence-forced refinement operation.

C.2 Methods that refine without local repair

Decision trees, online trees, Hoeffding trees, adaptive trees, extremely fast decision trees, and Mondrian forests refine leaves using information gain, variance reduction, or streaming split tests [BFOS84, DH00, BG09, MWS18, LRT14]. These are the closest algorithmic relatives of DGM's refinement test. The difference is structural: trees store distinctions in a rooted hierarchy, while DGM stores local pairwise distinction edges between runtime concepts and attaches repairable memories to the resulting nodes.

Adaptive Resonance Theory, including ART 1 for binary patterns and ART 2 for analog input patterns, addresses stable incremental category creation under mismatch and vigilance [CG87b, CG87a, Gro13]. Prototype methods, LVQ, SOMs, neural gas, growing neural gas, and Prototypical

Networks represent regions or classes by stored centers or exemplars [Koh90, Koh95, MS91, Fri95, SSZ17]. DGM shares the vocabulary of category creation and prototypes, but its concepts are operational hidden-state concepts whose edges are consequence-forced binary generators. The node is not only a region; it is a place where local memory can be repaired after the distinction is accepted.

Concept Bottleneck Models and Formal Concept Analysis emphasize explicit concept structures [KNT+20, GW99]. DGM does not claim human-interpretable concepts by default. Its concepts are runtime concepts: hidden-state anchors, centroids, local memories, and pairwise distinctions created from prediction consequences.

These methods solve the refinement half in various forms. They typically do not pair refinement with graph-gated neural local memories that continue to be repaired after the split.

C.3 Methods that route without runtime concept creation

Mixture-of-experts, Switch Transformers, and Product-Key Memory route tokens or queries to sparse trained capacity [SMM+17, FZS22, LSR+19]. Memory Networks, End-to-End Memory Networks, Neural Turing Machines, and Differentiable Neural Computers learn external read/write controllers [WCB15, SWF+15, GWD14, GWR+16]. These systems are important precedents for sparse reads and external memory. DGM differs in write semantics: the routed objects are runtime-created concepts whose edges generate an information structure and whose memories are repaired locally.

Continual learning studies forgetting, replay, regularization, and capacity growth across task sequences [KPR+17, LPR17, RRD+16, YYLH18]. DGM is complementary: it is an instance-level runtime data structure for local concept growth and memory repair, not a complete task-sequence continual learning system.

C.4 DGM's position

DGM combines refinement and repair but keeps them distinct:

$$\Delta_{\text{refine}} \text{ dominates} \Rightarrow \text{refine}, \qquad \Delta_{\text{repair}} \text{ dominates} \Rightarrow \text{repair}.$$

The measure-theoretic formalism follows standard sigma-algebra and conditional-expectation language [Bil95, Kal02]. The Bayes-risk order is related to Blackwell comparison and statistical decision theory [Bla53, Ber85]. The log-loss and squared-loss value identities are standard information-theoretic and Hilbert-space facts [CT06, GR07]. The contribution is to place these facts inside a runtime-learning model in which some errors are repair signals and others are evidence of missing distinctions.

D DGM-Embedding: Architectural Extension for Language Models

For language modeling, the world state is a prefix and the consequence is the next observed token. This makes the refine-or-repair decision concrete. If a prefix type is already represented, the layer should repair its next-token memory. If the same runtime concept is repeatedly asked to predict incompatible next-token consequences, the layer should refine the concept graph. This appendix gives DGM-Embedding as an architectural path, not as a completed large-scale language-model result.

The layer is the language-model analogue of the finite DGM loop. The prefix hidden state chooses edge-gated concepts; their memories produce a calibrated next-token distribution mixed

with the backbone; the true next token then supplies the detached consequence used for repair or refinement. There is no inference-time inner optimization problem in this write path.

D.1 Hidden-state concepts

Let a frozen or slowly trained backbone produce hidden states

$$h_t = \phi_\theta(x_{\leq t}),$$

and let

$$q_t = Qh_t \in \mathbb{R}^{d_q}$$

be the DGM query. A DGM layer maintains frozen query-space anchors $a_j \in \mathbb{R}^{d_q}$, movable query-space centroids $c_j \in \mathbb{R}^{d_q}$, sparse next-token count memories $n_{j,y}$ with totals n_j, and distinction edges:

$$\mathcal{S}_t^{\text{emb}} = \left(\{a_j, c_j, (n_{j,y})_{y\in\text{supp}(j)}, n_j\}_{j=1}^{N_t}, G_t\right).$$

The formal information structure is generated by anchors and edges. Retrieval uses centroids as adaptive statistics, while edge generators are computed from anchors. The recall-oriented candidate set $N(q_t)$ is the same one used by DGM: centroid neighbors for adaptive locality, anchor neighbors for formal route recall, and graph neighbors for recent conflicts.

D.2 Calibrated memory distribution

Let $p_0(y \mid x_{\leq t})$ be the backbone next-token distribution. For concept j, define the smoothed local distribution

$$p_j(y \mid x_{\leq t}) = \begin{cases} \dfrac{n_{j,y} + \alpha_0 p_0(y \mid x_{\leq t})}{n_j + \alpha_0}, & y \in \text{supp}(j), \\ \dfrac{\alpha_0 p_0(y \mid x_{\leq t})}{n_j + \alpha_0}, & y \notin \text{supp}(j), \end{cases}$$

where $\text{supp}(j)$ is the sparse set of tokens observed in concept j. The routed memory distribution and final prediction are

$$p_{\text{mem}}(y \mid x_{\leq t}) = \sum_{j\in N(q_t)} \alpha_j(q_t) p_j(y \mid x_{\leq t}),$$
$$p_{\text{final}}(y \mid x_{\leq t}) = (1-\lambda_t)p_0(y \mid x_{\leq t}) + \lambda_t p_{\text{mem}}(y \mid x_{\leq t}), \qquad \lambda_t \in [0,1].$$

The mixture is normalized without storing full-vocabulary counts at each concept. A learned or online-selected trust weight λ_t controls how much the memory layer can move the backbone distribution; for example, Hedge over a small grid of λ values gives regret to the best fixed trust weight in hindsight.

D.3 Read before observe

DGM-Embedding uses the same streaming order during training and inference whenever external consequences are available:

$$\text{predict } x_{t+1} \to \text{observe } x_{t+1} \to \text{repair or refine} \to \text{predict } x_{t+2}.$$

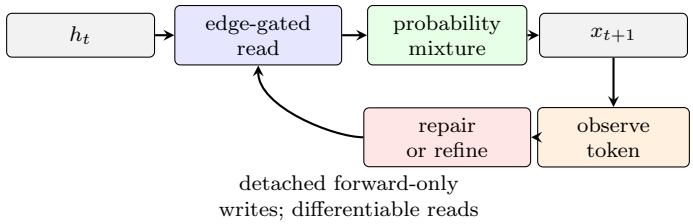

Figure 2: DGM-Embedding sequence update. The layer reads before the next token is known, then uses the observed next token as evidence for repair or refinement.

The order is important: the layer must predict before it sees the token that will update memory. After the next token is observed, its count is the detached sufficient-statistic write for every selected $j \in N(q_t)$, with all unselected memories unchanged:

$$n^{+}_{j,x_{t+1}} = n_{j,x_{t+1}} + \alpha_j(q_t), \qquad n^{+}_{j} = n_j + \alpha_j(q_t).$$

Hard-update mode is the special case $\alpha_{j^\star} = 1$ and $\alpha_j = 0$ for $j \neq j^\star$. If the penalized refinement test in (14) accepts a new concept, the layer creates a new anchor and centroid at q_t, initializes its sparse count memory with x_{t+1}, and adds distinction edges to $\text{Confused}_q(q_t, x_{t+1})$. Ordinary repairs may update centroids and memories but leave anchors fixed.

During streaming prediction, document continuation, user correction, tool feedback, or supervised next-token evaluation, the next observed token is external evidence and can refine or repair the runtime memory. During unconstrained generation, the model should not update from its own sampled token by default, because self-generated samples are actions, not observed consequences.

Algorithm 3 ObserveNextToken for DGM-Embedding

Input: prefix $x_{\leq t}$, observed token x_{t+1}, DGM state $\mathcal{S}_t$
$h_t \leftarrow \phi_\theta(x_{\leq t})$ and $q_t \leftarrow Qh_t$
retrieve $N(q_t)$ and compute the calibrated mixture p_{final}
$j^\star \leftarrow$ highest-weight concept
$\rho \leftarrow -\log p_{\text{final}}(x_{t+1} \mid x_{\leq t})$
if $\rho \leq \tau_{\text{repair}}$ **then**
    update DGM-Embedding memories for all $j \in N(q_t)$ using soft credit $\alpha_j(q_t)$
else
    accept a new anchored concept only if the penalized refinement gain test holds
end if
Hard variant: set $\alpha_{j^\star} = 1$ and $\alpha_j = 0$ for $j \neq j^\star$
return updated state

D.4 Sequential detached training

The update is detached:

$$\text{stopgrad}(\mathcal{S}_{t+1}) = \text{DGMObserve}(\text{stopgrad}(\mathcal{S}_t), h_t, x_{t+1}).$$

In sequential detached training, the model trains slow parameters through the read path and learns fast state through the same observation rule used at inference. The layer is not a replacement for backpropagation through the backbone. It is a replacement for inference-time and history-time backpropagation through the adaptive state.

Algorithm 4 SequentialDetachedTrainStep for a DGM-Embedding Transformer layer

Input: token block $x_{1:L}$, slow parameters Θ, DGM state $\mathcal{S}$
$\mathcal{L} \leftarrow 0$
for $t = 1, \ldots, L$ **do**
  compute hidden state h_t using the backbone
  $q_t \leftarrow Qh_t$
  retrieve $N(q_t)$ from the concept bank
  compute edge-gated weights $\alpha_j(q_t)$
  apply the DGM-Embedding read
  produce logits for x_{t+1} and accumulate $\mathcal{L} \leftarrow \mathcal{L} + \ell_t$
  $\mathcal{S} \leftarrow \text{stopgrad}(\text{DGMObserve}(\text{stopgrad}(\mathcal{S}), h_t, x_{t+1}))$
end for
update slow parameters Θ by backpropagation through the accumulated selected read computations
return updated Θ and $\mathcal{S}$

Block-lagged approximation. For throughput, an implementation may instead hold $\mathcal{S}$ fixed during the forward pass for a whole block, backpropagate through that block, and then apply detached DGMObserve updates for $t = 1, \ldots, L$. This block-lagged DGM training is more parallel, but it is an approximation: token $t + 1$ in the block does not see the state update produced after token t. It therefore introduces a train/inference mismatch and should not be described as the same observation program as streaming inference.

The main architectural claim stops here: DGM-Embedding is a next-token memory layer with differentiable reads and detached writes. Training modes for representation drift, the optional adapter variant, systems notes, and closed-form ridge repair are implementation extensions rather than the paper's central object; we give them in Appendix E.

E DGM Implementation Extensions

This appendix collects DGM mechanics and large-model details that are useful for implementations but secondary to the main refine-or-repair argument.

E.1 Trees as a hard-routing special case

A tree stores distinctions as a hierarchy. A mistake near the root changes every downstream routing decision. A graph stores distinctions locally between concepts. Creating a new concept does not rewrite old paths; it adds a local alternative and edges explaining why the new concept differs from its neighbors. Trees are still contained in the view below, but as a constrained hard-routing case.

Proposition E.1 (Trees as generalized hard-DGMs)**.** *Given a binary distinction tree, construct one generalized hard-route DGM concept for each leaf and attach the path distinctions as hard eligibility constraints for that concept. If the generalized hard route selects the unique eligible leaf concept and each concept memory equals the tree leaf predictor, then the generalized DGM and the tree make identical predictions.*

Thus the distinction-graph view contains trees after allowing arbitrary concept-level eligibility constraints. A decision tree is the sparse case in which distinctions are forced into a rooted hierarchy and routing is hard and single-path.

E.2 DGM data-structure cost

Let

$$Q(h) = \sum_{j \in N(h)} |\mathrm{Inc}_t(j)|$$

be the number of incident edge gates evaluated for the selected candidates. With exact nearest-neighbor search, prediction costs

$$O(N_t d + Q(h) + |N(h)|\, d + C_{\mathrm{local}}),$$

where C_{local} is the cost of evaluating the selected local memories. With approximate nearest-neighbor indexing, the $N_t d$ term can be replaced by the query cost of the index [IM98, JDJ19]. Creating an anchor-centroid pair costs $O(d)$ plus the cost of initializing its memory. Adding q distinction edges costs $O(qd)$. If each concept stores a sparse categorical memory with average support size $\bar{s}$, runtime memory is

$$O(N_t(2d + \bar{s}) + |E_t|\, d).$$

An embedding-mean variant instead uses $O(N_t(2d + d_e) + |E_t|\, d)$ memory, but it is not a calibrated log-loss memory without an additional probability model.

Theorem E.2 (DGM data-structure bound)**.** *For exact search over N_t centroids in $\mathbb{R}^d$, one edge-gated DGM prediction costs $O(N_t d + Q + |N|\, d + C_{\mathrm{local}})$, where $N = N(h)$ and $Q = Q(h)$. One observation that repairs a selected concept costs this plus the guarded local repair scoring cost. One observation that creates a new anchor-centroid pair and q distinction edges costs an additional $O(d+qd)$. With sparse categorical memories of average support $\bar{s}$, the runtime state uses $O(N_t(2d+\bar{s}) + |E_t|\, d)$ memory.*

E.3 Ridge local memory

For few-shot adaptation or local readout repair, a memory stores support embeddings $X_{\mathrm{sup}} \in \mathbb{R}^{n \times d}$ and labels $Y \in \mathbb{R}^{n \times C}$. The closed-form adapter is the dual ridge/Tikhonov solution [TA77, HK70]:

$$\Gamma = (X_{\mathrm{sup}} X_{\mathrm{sup}}^\top + \lambda I)^{-1} Y, \qquad \mathrm{logits}(z) = z X_{\mathrm{sup}}^\top \Gamma.$$

Projection Memory and CPM are advanced repair rules inside M_j after the graph has fixed the relevant local concept; Appendix H treats that relation directly.

E.4 Training modes for representation drift

Anchors, centroids, and edges live in query space. If the encoder or query map changes rapidly while the DGM state persists, old anchors and edges describe stale geometry. Reports should therefore state which training mode is used.

Mode A: frozen encoder. Train the backbone and query map first, then freeze ϕ_θ, Q, and the backbone distribution used for DGM smoothing while running DGM runtime learning. This is the cleanest protocol for the first experiments and is the default interpretation of the theory.

Mode B: EMA query encoder. Maintain a slowly moving query encoder

$$\bar{\theta} \leftarrow \beta\bar{\theta} + (1-\beta)\theta,$$

and create anchors, centroids, and edge generators in the query space of $\phi_{\bar{\theta}}$ and $Q_{\bar{\theta}}$. The backbone can still train, but the DGM state is tied to the slower geometry.

Mode C: periodic state rebuild. During end-to-end training, rebuild or clear the DGM state every K optimization steps, or rebuild it from a replay buffer under the current query map. The formal guarantees then apply within a rebuild interval. Cross-interval state carryover is a systems approximation, not a claim of anchor-stable refinement.

E.5 Non-canonical adapter extension

Beyond the categorical next-token memory in Appendix D, a richer optional layer stores a local linear repair map. It requires an explicit residual target and should not be conflated with the calibrated next-token memory mechanism:

$$\mathcal{S}_t^{\text{adp}} = \{a_j, c_j, W_j, n_j\}_{j=1}^{N_t} \cup G_t, \qquad W_j \in \mathbb{R}^{d\times r}.$$

Let $K : \mathbb{R}^d \to \mathbb{R}^r$ be a learned or fixed key projection and set $k_t = Kh_t$. The adapter read is

$$\Delta h_t = \sum_{j\in N(q_t)} \alpha_j(q_t) W_j k_t, \qquad \tilde{h}_t = h_t + \gamma_\theta(h_t)\Delta h_t.$$

Here $W_j k_t$ is a hidden-space residual, so the write rule requires an explicit residual target $v_t \in \mathbb{R}^d$. Let $P_{\text{emb}\to h} : \mathbb{R}^{d_e} \to \mathbb{R}^d$ be a chosen projection from output-embedding space to hidden-residual space. Natural choices include a token-embedding target $P_{\text{emb}\to h} e_{x_{t+1}}$, a teacher-forced output-error target $P_{\text{emb}\to h}(e_{x_{t+1}} - \hat{e}_t)$, or a distillation target $h_t^{\text{teacher}} - h_t$. Without such a target, the adapter is only an architectural extension.

Given a local residual target, the projected rank-one repair for a selected concept is the same residual-correction pattern that appears in passive-aggressive updates, RLS/ridge-style online least squares, and delta-rule fast-weight memories [CDK+06, Vov97, SIS21]:

$$W_j^+ = W_j + \frac{\eta\alpha_j(q_t)}{1 + \eta\alpha_j(q_t)\,\|k_t\|_2^2}(v_t - W_j k_t)k_t^\top.$$

Equivalently, a memory may maintain small ridge sufficient statistics and solve the local adapter in closed form [TA77, HK70].

E.6 Cost and systems notes

During sequential detached training, backpropagation sees the selected read computation, while the DGM state is updated in the same order as streaming inference. The important comparison is with test-time gradient adaptation. A TTT-style layer typically performs an inner backward pass at inference time. A DGM layer performs retrieval, sparse local read, and closed-form or sufficient-statistic repair. The cost is forward-only:

$$\text{DGM inference} = \text{backbone forward} + \text{retrieval} + \text{sparse local repair}.$$

Theorem E.3 (DGM layer cost)**.** *Consider a DGM adapter extension with hidden dimension d, query dimension d_q, local rank r, candidate count $|N|$, selected incident edge-gate count Q_N, and concept-bank query cost $C_{\text{ret}}(N_t, d_q)$. For one token, the read overhead is*

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N + |N|\, dr).$$

If the write path is detached and uses rank-one local repair, the write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N + |N|\,(d_q + dr)).$$

For a block of length L, the DGM state history need not be stored for backpropagation; the additional activation memory is the selected read computation, not the full sequence of memory states. The categorical DGM-Embedding layer replaces the $|N|\, dr$ adapter term by sparse count lookup, edge-gated mixture, and trust-weight update costs.

The concept bank can be sharded by the same principle used in retrieval and sparse expert systems. Each device owns a subset of concepts. A query is routed to a small set of shards, each shard returns local top-k candidates, and a final top-k merge selects the memories used by the read path. Only selected memories receive gradients. Detached writes are sent back to the owning shards.

Budgeting is necessary for long training. A practical DGM layer maintains $N_t \leq N_{\max}$, merges only compatible unprotected concepts, and otherwise evicts or compresses low-score memories. These budget actions are maintenance operations, not learning operations in the formal refine-or-repair sense. The refine-or-repair theorems apply between maintenance events; merge, eviction, and compression may destroy information and must be evaluated as resource-management approximations.

E.7 Closed-form local repair

Ridge memory is the main closed-form adapter used in our checks. Let $X_{\text{sup}} \in \mathbb{R}^{n \times d}$ be normalized support embeddings stored by a DGM memory and let $Y \in \mathbb{R}^{n \times C}$ be one-hot consequences. The adapter computes the dual ridge/Tikhonov solution [TA77, HK70]

$$\Gamma = (X_{\text{sup}} X_{\text{sup}}^\top + \lambda I)^{-1} Y, \qquad \text{logits}(z) = z X_{\text{sup}}^\top \Gamma.$$

This is projection-style repair: it fits the local adapter in one linear solve rather than by iterated backpropagation. In a large model, the same idea can be applied per concept, per document, or per retrieved shard after the relevant distinction structure has been fixed.

F Additional Computational Checks

This appendix reports secondary checks that are useful for debugging implementations but are not the main evidence for the refine-or-repair claim.

F.1 Online image streams

We run small online image-stream checks on MNIST and CIFAR-10 using cached local datasets. These are sanity checks, not evidence that DGM is a competitive image classifier. In these experiments the DGM memory uses exemplar repair. The online backpropagation baseline sees each example once and performs one gradient step. Offline backpropagation is a batch upper bound.

The relevant future settings are context switching, few-shot adaptation with a frozen encoder, long-document memory, repeated syntax regimes, and hidden task identity.

Dataset	Offline backprop	Online backprop	Online DGM memory
MNIST	0.843 ± 0.009	0.682 ± 0.038	0.851 ± 0.003
CIFAR-10	0.273 ± 0.003	0.137 ± 0.010	0.243 ± 0.008

Table 5: Image-stream sanity checks. The point is not to beat fully trained image models, but to compare one-pass runtime updates.

F.2 Frozen-embedding adapter check

For a stronger adapter benchmark, we train a shared convolutional encoder and freeze it. The adaptation problem is deliberately narrow: given only five labeled support examples per class, adapt the readout. We compare nearest-neighbor memory, ridge memory as a closed-form projection adapter from Appendix E, a one-pass backpropagation head, and a 200-step backpropagation head on the same frozen embeddings.

Dataset	Method	Accuracy	Adapter time
MNIST, 5-shot	one-pass backprop head	0.100 ± 0.000	0.0002s
MNIST, 5-shot	200-step backprop head	0.332 ± 0.009	0.0320s
MNIST, 5-shot	kNN memory	0.486 ± 0.005	0.0006s
MNIST, 5-shot	ridge memory	$\mathbf{0.623 \pm 0.003}$	0.0018s
CIFAR-10, 5-shot	one-pass backprop head	0.115 ± 0.014	0.0003s
CIFAR-10, 5-shot	200-step backprop head	0.359 ± 0.009	0.0325s
CIFAR-10, 5-shot	kNN memory	0.355 ± 0.005	0.0006s
CIFAR-10, 5-shot	ridge memory	$\mathbf{0.397 \pm 0.010}$	0.0004s

Table 6: Few-shot adapter check on frozen embeddings. Ridge memory fits the adapter by a closed-form projection solve.

F.3 Projection-memory residual identity

We verify the projection-memory residual contraction identity used by local repair. For random W, k, v and step size η, the soft Projection Memory update satisfies

$$v - W_{t+1}k = \frac{1}{1 + \eta \|k\|_2^2}(v - W_t k).$$

Over 200 random trials, the maximum absolute error between the measured residual ratio and the formula was $3.33 \cdot 10^{-16}$.

G Proof Sketches

This section collects the main proof ideas in the order used by the paper: runtime excess risk decomposes into Δ_{repair} and Δ_{refine}; repair-only learners have linear regret against refined oracles; useful refinements are exactly those that change the Bayes action; minimal refinement adds the missing distinction and nothing else; useful distinctions have exact value under standard losses; and guarded DGM updates decrease the local penalized objective. The measure-theoretic notation,

conditional-expectation facts, and decision-theoretic Bayes-risk framing are standard [Bil95, Kal02, Bla53, Ber85]. Full formal versions appear in Appendices I, J, and K.

Theorem G.1 (Runtime error decomposition, proof sketch for Theorem 2.2)**.** *The statement is Theorem 2.2.*

Proof sketch. Add and subtract the Bayes risk under the current information:

$$R_L(f_t) - R_L(\Pi^\star) = \Delta_{\text{repair}}(f_t, \Pi_t) + \Delta_{\text{refine}}(\Pi_t, \Pi^\star).$$

The first term is nonnegative because f_t is one of the Π_t-measurable predictors over which $R_L(\Pi_t)$ optimizes. The second is nonnegative because a refinement enlarges the admissible predictor class. □

Theorem G.2 (Repair-only regret, proof sketch for Theorem 2.3)**.** *The statement is Theorem 2.3.*

Proof sketch. Condition on the history before round t. A repair-only learner's prediction is Π-measurable, so its conditional expected loss is at least $R_L(\Pi)$ by definition of Bayes risk under the partition. Summing over rounds gives cumulative expected loss at least $TR_L(\Pi)$. A refined oracle using $\Pi^\star$ pays $TR_L(\Pi^\star)$, so the regret lower bound follows by subtraction. □

Theorem G.3 (Loss-relevant refinement, proof sketch for Theorem J.1)**.** *The statement is Theorem 2.6.*

Proof sketch. If the $\mathcal{G}$-Bayes action is already $\mathcal{F}$-measurable, then it is feasible under the old information and attains the refined risk, so the refinement gap is zero. Conversely, if the old and refined risks are equal, any $\mathcal{F}$-Bayes action is also $\mathcal{G}$-optimal. Almost sure uniqueness of the $\mathcal{G}$-Bayes action forces equality with the refined Bayes action, so the refined action has an $\mathcal{F}$-measurable version. □

Theorem G.4 (Contradiction, proof sketch for Theorem I.3)**.** *The statement is Corollary 2.5.*

Proof sketch. If f is $\mathcal{P}$-measurable and ω, ω' lie in the same atom, then

$$f(\omega) = f(\omega').$$

If $f(\omega) = y(\omega)$ and $f(\omega') = y(\omega')$, then

$$y(\omega) = y(\omega'),$$

contradicting the hypothesis. □

Theorem G.5 (Minimal finite refinement, proof sketch for Theorem I.1)**.** *The statement is Theorem 2.7.*

Proof sketch. Every atom of $\mathcal{P}_D$ lies inside an atom of $\mathcal{P}$, so $\mathcal{P}_D$ refines $\mathcal{P}$. Also

$$D = \bigcup_{B \in \mathcal{P}} (B \cap D),$$

so D is $\mathcal{P}_D$-measurable. If $\mathcal{Q}$ refines $\mathcal{P}$ and makes D measurable, then every atom $Q \in \mathcal{Q}$ satisfies

$$Q \subseteq B$$

for some $B \in \mathcal{P}$, and also either $Q \subseteq D$ or $Q \subseteq D^c$. Hence

$$Q \subseteq B \cap D \qquad \text{or} \qquad Q \subseteq B \cap D^c.$$

Thus $\mathcal{Q}$ refines $\mathcal{P}_D$. □

Theorem G.6 (Minimal sigma-algebra refinement, proof sketch for Theorem I.2)**.** *The statement is Theorem 2.8.*

Proof sketch. By definition,

$$\mathcal{F} \vee \sigma(U) = \sigma(\mathcal{F} \cup \sigma(U)).$$

This sigma-algebra contains $\mathcal{F}$ and makes U measurable. If $\mathcal{G}$ also contains $\mathcal{F}$ and makes U measurable, then

$$\mathcal{F} \cup \sigma(U) \subseteq \mathcal{G}.$$

Since $\mathcal{G}$ is a sigma-algebra,

$$\sigma(\mathcal{F} \cup \sigma(U)) \subseteq \mathcal{G}.$$

□

Theorem G.7 (Risk monotonicity, proof sketch for Theorem J.2)**.** *The statement is Theorem 3.1.*

Proof sketch. If $\mathcal{F} \subseteq \mathcal{G}$, then every $\mathcal{F}$-measurable action is $\mathcal{G}$-measurable. Therefore

$$\{a : a \text{ is } \mathcal{F}\text{-measurable}\} \subseteq \{a : a \text{ is } \mathcal{G}\text{-measurable}\}.$$

Taking the infimum of the same expected loss over the larger set gives

$$R_L(\mathcal{G}) \leq R_L(\mathcal{F}).$$

□

Theorem G.8 (Log-loss value, proof sketch for Theorem J.3)**.** *The statement is Theorem 3.2.*

Proof sketch. For any $\mathcal{F}$-measurable conditional distribution q, the standard log-loss decomposition gives

$$\mathbb{E}[-\log q(Y)] = H(Y \mid \mathcal{F}) + \mathbb{E}\left[D_{\mathrm{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{F}) \| q(\cdot))\right].$$

The minimizer is $q^\star = \mathbb{P}(Y \in \cdot \mid \mathcal{F})$, so $R_{\log}(\mathcal{F}) = H(Y \mid \mathcal{F})$. Similarly $R_{\log}(\mathcal{G}) = H(Y \mid \mathcal{G})$. Therefore

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = H(Y \mid \mathcal{F}) - H(Y \mid \mathcal{G}) = I(Y; \mathcal{G} \mid \mathcal{F}).$$

□

Theorem G.9 (Squared-loss value, proof sketch for Theorem J.5)**.** *The statement is Theorem 3.4.*

Proof sketch. The optimal $\mathcal{F}$-predictor is $\mathbb{E}[Y \mid \mathcal{F}]$, the standard L^2 projection. Since $\mathcal{F} \subseteq \mathcal{G}$,

$$Y - \mathbb{E}[Y \mid \mathcal{F}] = (Y - \mathbb{E}[Y \mid \mathcal{G}]) + (\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]).$$

The two terms are orthogonal because the first has conditional mean zero given $\mathcal{G}$ and the second is $\mathcal{G}$-measurable. Taking squared ℓ_2 norms and expectations gives the identity. □

Theorem G.10 (Anchor edge separation, proof sketch for Theorem K.1)**.** *The statement is Theorem 4.1.*

Proof sketch. With $u = (a_i - a_j)/\|a_i - a_j\|_2$ and $b = u^\top(a_i + a_j)/2$,

$$u^\top a_i - b = \frac{1}{2}u^\top(a_i - a_j) = \frac{1}{2}\|a_i - a_j\|_2,$$
$$u^\top a_j - b = -\frac{1}{2}u^\top(a_i - a_j) = -\frac{1}{2}\|a_i - a_j\|_2.$$

□

Theorem G.11 (DGM graph partition, proof sketch for Theorem K.2)**.** *The statement is Theorem 4.2.*

Proof sketch. The graph signature records the value of every binary edge generator. Equality of signatures is an equivalence relation, and its classes are exactly the nonempty common level sets of those generators. Adding a new edge appends one binary coordinate to the signature, so each old atom is either left intact or cut by the two events $g^{-1}(1)$ and $g^{-1}(0)$. □

Theorem G.12 (Hard-route measurability, proof sketch for Theorem K.3)**.** *The statement is Theorem 4.3.*

Proof sketch. Edge eligibility is determined by finitely many edge-bit events, so it is measurable with respect to $\mathcal{F}_t^{\text{edge}}$. The anchor-order sigma-algebra generated by all pairwise anchor comparisons determines the nearest anchor in any eligible subset under deterministic tie-breaking. The exact hard selected index is a finite composition of these two finite decisions, so it is measurable with respect to their join. Movable centroids can still be used for the soft retrieval path, but they are not part of this formal information structure. □

Theorem G.13 (Soft routing hard limit, proof sketch for Theorem K.4)**.** *The statement is Theorem 4.5.*

Proof sketch. The sparse soft read can only assign mass to candidates it retrieved. Therefore, if the exact hard-route concept is omitted from the candidate set, no temperature limit can recover that route. Under candidate recall, the remaining argument is a finite softmax and gate-sharpening calculation. For ordinary nearest-center routing, every nonnearest candidate has unnormalized weight smaller by at least $\exp(-\delta/T)$, so the total nonnearest mass is bounded by a finite candidate-count factor times this exponential. For a structural edge gate with uniformly positive weight, a wrong-side signed margin $-\delta_e$ contributes a log-sigmoid term bounded by $-\delta_e/\tau_e$. Degree normalization rescales the exponent but does not remove exponential decay under a uniform degree bound. After normalizing against an eligible candidate with compatibility bounded below and bounded distance-score ratio, the wrong-side mass vanishes as the edge temperature tends to zero. □

Theorem G.14 (Refine-or-repair descent, proof sketch for Theorem K.5)**.** *The statement is Theorem 4.7.*

Proof sketch. Guarded repair proposes local memory or centroid changes while keeping anchors and edges fixed, then commits only candidates that do not increase empirical loss on the fixed scoring buffer $\mathcal{B}_t^{\text{score}}$. The identity candidate is always feasible. The concept and edge penalties therefore stay fixed, so the local objective $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. A refinement that creates one anchored concept and q edges changes $J_{\mathcal{B}_t^{\text{score}}}$ by the empirical-risk decrease on the scoring buffer minus $\lambda_{\text{concept}} + \lambda_{\text{edge}} q$, which is exactly the penalized refinement gain in (13). The acceptance test requires this net decrease to exceed the empirical repair gain plus $\tau + 2\epsilon_m$. □

Theorem G.15 (Safe refinement, proof sketch for Theorem K.6)**.** *The statement is Theorem 4.8.*

Proof sketch. At each time t, condition on the filtration containing the past, current DGM state, and proposal buffer. The predictable finite candidate class is then fixed, while the scoring buffer remains independent by assumption. Hoeffding's inequality bounds the deviation between empirical and population risk for the old state and for each candidate-refined state [Hoe63]. A union bound over the finite class gives failure probability of order δ/t^2 after using the displayed ϵ_t, and a second union bound over all times is summable. Each gain is a difference of two risks, so its empirical-population deviation is at most twice the risk deviation. If the empirical gain exceeds cost plus $\tau + 2\epsilon_t$, the population gain exceeds cost plus τ on this high-probability event. □

Corollary G.16 (Bounded LM score, proof sketch for Corollary K.7)**.** *The statement is Corollary 4.9.*

Proof sketch. Clipping and normalizing next-token negative log likelihood gives a score in $[0, 1]$. The adaptive refinement theorem therefore applies directly to the clipped scoring rule. The conclusion is intentionally about clipped population gain, not raw unbounded cross-entropy. □

Theorem G.17 (Anchor-stable DGM information structure, proof sketch for Theorem K.8)**.** *The statement is Theorem 4.10.*

Proof sketch. Every edge generator $g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}$ is a deterministic function of the two frozen endpoint anchors. A repair step changes neither endpoint anchor, so it changes neither u_e nor b_e. It also leaves every anchor-distance comparison event unchanged. Therefore the anchor-generated DGM information structure is unchanged, even though the retrieval centroids and local memories may be repaired. □

Corollary G.18 (Finite refinements, proof sketch for Corollary K.9)**.** *The statement is Corollary 4.11.*

Proof sketch. After m accepted refinements a common potential Φ has decreased by at least $m\tau$. Since the potential is nonnegative, $m\tau \leq \Phi(\mathcal{S}_0)$. This argument requires a common potential; changing local scoring buffers do not by themselves define a global finiteness bound. □

Proposition G.19 (Tree as generalized hard-DGM, proof sketch for Proposition K.10)**.** *The statement is Proposition E.1.*

Proof sketch. Make one concept for each tree leaf and attach the root-to-leaf binary tests as hard eligibility constraints. Every input satisfies exactly the path constraints of the leaf reached by the tree, so the generalized hard route selects the same memory and returns the same prediction. □

Theorem G.20 (DGM data structure, proof sketch for Theorem K.11)**.** *The statement is Theorem E.2.*

Proof sketch. A prediction computes centroid and anchor distances from h to N_t concepts in $\mathbb{R}^d$, costing

$$O(N_t d).$$

It then evaluates $Q(h)$ selected incident edge gates and $|N(h)|$ selected local memories, costing $O(Q(h) + |N(h)|\, d + C_{\text{local}})$ including sparse routing weights. Guarded repair adds the cost of

scoring proposed local candidates. Creating one anchor-centroid pair stores two vectors in $\mathbb{R}^d$, still $O(d)$. Adding q distinction edges stores q vectors and thresholds, costing

$$O(qd).$$

For mean memories, each concept stores $a_j, c_j \in \mathbb{R}^d$ and $r_j \in \mathbb{R}^{d_e}$, while each edge stores one distinction vector in $\mathbb{R}^d$. Hence memory is

$$O(N_t(2d + d_e) + |E_t|\, d).$$

□

Theorem G.21 (DGM layer cost, proof sketch for Theorem K.12)**.** *The statement is Theorem E.3.*

Proof sketch. Computing $q = Qh$ costs $O(dd_q)$. Retrieval costs $C_{\text{ret}}(N_t, d_q)$ by definition. Edge-gate evaluation over selected incident edges costs $O(Q_N)$. For each selected concept, a rank-r adapter computes

$$W_j K h.$$

The matrix-vector product with one local adapter costs $O(dr)$, so $|N|$ selected adapters cost

$$O(|N|\, dr).$$

The detached write path repeats query, retrieval, and edge-gate evaluation, then updates only selected local states. A rank-one local repair or low-rank sufficient-statistic update costs $O(dr)$ per selected memory, with $O(d_q)$ additional concept bookkeeping. Hence write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N + |N|\,(d_q + dr)).$$

Because the write path is detached, backpropagation does not need intermediate copies of $\mathcal{S}_t$ for all t. It stores the selected read computation used in the forward loss. □

Theorem G.22 (One-hot memory, proof sketch for Theorem K.13)**.** *The statement is Theorem H.1.*

Proof sketch. If $\omega, \omega' \in B_i$, then both one-hot features equal e_i, so

$$W\phi_{\mathcal{P}}(\omega) = We_i = W\phi_{\mathcal{P}}(\omega').$$

Thus $W\phi_{\mathcal{P}}$ is constant on atoms. Conversely, if f is constant on each B_i, put that constant value in column i of W. Then

$$f(\omega) = W\phi_{\mathcal{P}}(\omega)$$

for every ω. □

Theorem G.23 (CPM conflict, proof sketch for Theorem K.14)**.** *The statement is Theorem H.2.*

Proof sketch. If $\omega, \omega' \in B_i$, then

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

After committing $We_i = v(\omega)$, the later evidence $We_i = v(\omega')$ has the same key. Its innovation relative to the old key span is

$$z = (I - e_i e_i^\top)e_i = 0.$$

Its residual is

$$e = v(\omega') - We_i = v(\omega') - v(\omega) \neq 0.$$

□

H From Distinctions to Projection Memory

DGM uses refinement and repair in the same runtime object. Distinction learning asks what the learner can tell apart. Projection learning assumes such a representation and asks how a state inside it should be minimally repaired. The repair side is related to Bregman projection, passive-aggressive learning, online regression, and fast-weight memory updates [Bre67, CDK+06, Vov97, BHM+16, SIS21]. This section makes the relation formal in the finite one-hot case.

H.1 One-hot partition memory

Let $\mathcal{P} = \{B_1, \ldots, B_n\}$ be a finite partition and let $\phi_{\mathcal{P}} : \Omega \to \mathbb{R}^n$ be its one-hot feature map:

$$\phi_{\mathcal{P}}(\omega) = e_i \qquad \text{if} \qquad \omega \in B_i.$$

A memory matrix $W \in \mathbb{R}^{m \times n}$ defines a value map

$$f_W(\omega) = W\phi_{\mathcal{P}}(\omega).$$

This is exactly a table over the atoms of $\mathcal{P}$: every state inside the same atom receives the same value.

Theorem H.1 (One-hot partition memory)**.** *For every $W \in \mathbb{R}^{m \times n}$, the map $f_W(\omega) = W\phi_{\mathcal{P}}(\omega)$ is $\mathcal{P}$-measurable. Conversely, every $\mathcal{P}$-measurable map $f : \Omega \to \mathbb{R}^m$ is represented by some W through $f = f_W$.*

Thus a one-hot associative memory is not merely analogous to a partition. It is an exact finite representation of all vector-valued predictors measurable with respect to that partition.

H.2 CPM conflict as missing distinction

For old key matrices K, a new key k has CPM innovation

$$z = (I - KK^{\dagger})k. \tag{18}$$

Conservative Projection Memory detects conflict when a new key has zero innovation but nonzero residual. In the one-hot finite representation, this diagnostic is exactly the statement that two world states have been placed in the same atom despite requiring different values.

Theorem H.2 (CPM conflict equals missing distinction)**.** *Let $\omega, \omega' \in B_i$ for an atom $B_i \in \mathcal{P}$, and suppose required values satisfy $v(\omega) \neq v(\omega')$. Let $k = \phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i$. If $We_i = v(\omega)$ has already been committed, then evidence requiring $We_i = v(\omega')$ has CPM innovation $z = 0$ and residual $e \neq 0$.*

The theorem explains why a projection update should stop. The key is not new; the old information structure maps both states to the same coordinate. The residual is nonzero because the consequence differs. The missing update is a refinement of $\mathcal{P}$ that separates ω from ω'.

H.3 The hierarchy

The resulting hierarchy is

$$\text{distinction refinement} \supset \text{projection repair} \supset \text{implicit gradient} \supset \text{explicit gradient descent.}$$

Distinction refinement changes the sigma-algebra $\mathcal{F}_t$. Projection repair changes a state s_t inside a fixed information geometry. Gradient descent is the linearized ℓ_2 special case of soft projection, and backpropagation is the mechanism that computes such gradients in differentiable static graphs [RHW86].

This is not a rejection of gradient learning or projection memory. It is a separation of roles. When the representation is adequate, repair and optimization are the right tools. When the representation identifies states that consequences require it to separate, the first learning act is to create the missing distinction.

I Minimal Refinement Proofs

The statements in this appendix use standard facts about finite partitions, generated sigma-algebras, and measurability [Bil95, Kal02]. The contribution is the runtime-learning interpretation: refinement is the minimal information update needed by new evidence.

Theorem I.1 (Minimal finite refinement, formal version of Theorem 2.7)**.** *Let Ω be finite, let $\mathcal{P}$ be a partition of Ω, and let $D \subseteq \Omega$. Define*

$$\mathcal{P}_D = \{B \cap D : B \in \mathcal{P}, B \cap D \neq \varnothing\} \cup \{B \cap D^c : B \in \mathcal{P}, B \cap D^c \neq \varnothing\}.$$

Then:

- **Part 1.** *$\mathcal{P}_D$ is a partition of Ω.*
- **Part 2.** *$\mathcal{P}_D$ refines $\mathcal{P}$.*
- **Part 3.** *D is $\mathcal{P}_D$-measurable.*
- **Part 4.** *If $\mathcal{Q}$ refines $\mathcal{P}$ and D is $\mathcal{Q}$-measurable, then $\mathcal{Q}$ refines $\mathcal{P}_D$.*

Consequently, $\mathcal{P}_D$ is the unique minimal refinement of $\mathcal{P}$ making D measurable.

Proof. For each $B \in \mathcal{P}$, the nonempty sets among $B \cap D$ and $B \cap D^c$ are disjoint and have union B. Since the atoms of $\mathcal{P}$ are disjoint and cover Ω, the atoms of $\mathcal{P}_D$ are disjoint and cover Ω. Thus Part 1 holds.

Every atom of $\mathcal{P}_D$ is contained in some $B \in \mathcal{P}$, so Part 2 holds. Also

$$D = \bigcup_{B \in \mathcal{P}} (B \cap D),$$

where empty intersections are ignored. Hence D is a union of atoms of $\mathcal{P}_D$, proving Part 3.

Let $\mathcal{Q}$ refine $\mathcal{P}$ and make D measurable. For any atom $Q \in \mathcal{Q}$, there exists $B \in \mathcal{P}$ with

$$Q \subseteq B.$$

Since D is $\mathcal{Q}$-measurable, each atom of $\mathcal{Q}$ is either contained in D or disjoint from D. Hence

$$Q \subseteq B \cap D \qquad \text{or} \qquad Q \subseteq B \cap D^c.$$

Thus each atom of $\mathcal{Q}$ is contained in an atom of $\mathcal{P}_D$, so $\mathcal{Q}$ refines $\mathcal{P}_D$. This proves Part 4.

If another partition has the same minimal property, it refines $\mathcal{P}_D$ and is refined by $\mathcal{P}_D$, so the two partitions have the same atoms. □

Theorem I.2 (Minimal sigma-algebra refinement, formal version of Theorem 2.8)**.** *Let $(\Omega, \mathcal{E})$ be a measurable space, let $\mathcal{F} \subseteq \mathcal{E}$ be a sigma-algebra, and let $U : \Omega \to \mathcal{U}$ be a measurable map into a measurable space $(\mathcal{U}, \Sigma_{\mathcal{U}})$. The join $\mathcal{F} \vee \sigma(U)$ is the smallest sigma-algebra $\mathcal{G} \subseteq \mathcal{E}$ such that $\mathcal{F} \subseteq \mathcal{G}$ and U is $\mathcal{G}$-measurable.*

Proof. By definition,

$$\mathcal{F} \vee \sigma(U) = \sigma(\mathcal{F} \cup \sigma(U)).$$

Therefore $\mathcal{F} \subseteq \mathcal{F} \vee \sigma(U)$ and $\sigma(U) \subseteq \mathcal{F} \vee \sigma(U)$, so U is measurable with respect to $\mathcal{F} \vee \sigma(U)$.

Let $\mathcal{G}$ be any sigma-algebra such that $\mathcal{F} \subseteq \mathcal{G}$ and U is $\mathcal{G}$-measurable. The latter condition means $\sigma(U) \subseteq \mathcal{G}$. Hence

$$\mathcal{F} \cup \sigma(U) \subseteq \mathcal{G}.$$

Since $\mathcal{G}$ is a sigma-algebra,

$$\sigma(\mathcal{F} \cup \sigma(U)) \subseteq \mathcal{G}.$$

Thus $\mathcal{F} \vee \sigma(U) \subseteq \mathcal{G}$. □

Theorem I.3 (Contradiction requires distinction, formal version of Corollary 2.5)**.** *Let $I : \Omega \to \mathcal{Z}$ be an information map and let $y : \Omega \to \mathsf{A}$ be a required output. Suppose $I(\omega) = I(\omega')$ but $y(\omega) \neq y(\omega')$. Then no predictor $f = g \circ I$ satisfies both*

$$f(\omega) = y(\omega), \qquad f(\omega') = y(\omega').$$

Proof. Since $f = g \circ I$ and $I(\omega) = I(\omega')$,

$$f(\omega) = f(\omega').$$

If both requirements held, then

$$y(\omega) = f(\omega) = f(\omega') = y(\omega'),$$

contradicting $y(\omega) \neq y(\omega')$. □

J Value of Distinction Proofs

The proofs below use standard tools from statistical decision theory, information theory, proper scoring rules, and conditional expectation: Bayes risk under constrained information, log-loss optimality of the conditional law, and the L^2 projection identity for conditional expectation [Bla53, Ber85, CT06, GR07, Bil95, Kal02].

Theorem J.1 (Loss-relevant refinement, formal version of Theorem 2.6)**.** *Let $\mathcal{F} \subseteq \mathcal{G} \subseteq \mathcal{E}$. Assume Bayes actions exist under $\mathcal{F}$ and $\mathcal{G}$, and that the $\mathcal{G}$-Bayes action is almost surely unique. Then*

$$R_L(\mathcal{F}) - R_L(\mathcal{G}) = 0$$

if and only if $a^\star_{\mathcal{G}}$ has an $\mathcal{F}$-measurable version.

Proof. If $a^\star_{\mathcal{G}}$ is $\mathcal{F}$-measurable, then it is feasible for $R_L(\mathcal{F})$, so

$$R_L(\mathcal{F}) \le \mathbb{E}[L(Y, a^\star_{\mathcal{G}})] = R_L(\mathcal{G}).$$

The inclusion $\mathcal{F} \subseteq \mathcal{G}$ gives $R_L(\mathcal{G}) \le R_L(\mathcal{F})$, so the risks are equal. Conversely, if the risks are equal, let $a^\star_{\mathcal{F}}$ be an $\mathcal{F}$-Bayes action. It is also $\mathcal{G}$-measurable and achieves $R_L(\mathcal{G})$. By almost sure uniqueness of $a^\star_{\mathcal{G}}$, $a^\star_{\mathcal{F}} = a^\star_{\mathcal{G}}$ almost surely. Hence $a^\star_{\mathcal{G}}$ has an $\mathcal{F}$-measurable version. □

Theorem J.2 (Bayes-risk monotonicity, formal version of Theorem 3.1)**.** *Let Y be a target random variable, let A be an action space, and let L be an integrable loss. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_L(\mathcal{G}) \le R_L(\mathcal{F}).$$

Proof. Define

$$\mathcal{D}_{\mathcal{F}} = \{a : \Omega \to \mathsf{A} : a \text{ is } \mathcal{F}\text{-measurable}\}, \qquad \mathcal{D}_{\mathcal{G}} = \{a : \Omega \to \mathsf{A} : a \text{ is } \mathcal{G}\text{-measurable}\}.$$

Since $\mathcal{F} \subseteq \mathcal{G}$,

$$\mathcal{D}_{\mathcal{F}} \subseteq \mathcal{D}_{\mathcal{G}}.$$

Therefore

$$\begin{aligned} R_L(\mathcal{G}) &= \inf_{a \in \mathcal{D}_{\mathcal{G}}} \mathbb{E}[L(Y, a)] \\ &\le \inf_{a \in \mathcal{D}_{\mathcal{F}}} \mathbb{E}[L(Y, a)] \\ &= R_L(\mathcal{F}). \end{aligned}$$

□

Theorem J.3 (Log-loss value, formal version of Theorem 3.2)**.** *Let Y be finite and let $\mathcal{F} \subseteq \mathcal{G}$. Under log loss, an admissible action is a measurable conditional distribution $q(\cdot)$ on the range of Y, and the loss is $-\log q(Y)$. Then*

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = I(Y; \mathcal{G} \mid \mathcal{F}) = \mathbb{E}\left[D_{\mathrm{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{G}) \| \mathbb{P}(Y \in \cdot \mid \mathcal{F}))\right].$$

Proof. For any $\mathcal{F}$-measurable conditional distribution q, write $p_{\mathcal{F}}(y) = \mathbb{P}(Y = y \mid \mathcal{F})$. Then

$$\begin{aligned} \mathbb{E}[-\log q(Y) \mid \mathcal{F}] &= \sum_y p_{\mathcal{F}}(y)(-\log q(y)) \\ &= \sum_y p_{\mathcal{F}}(y)(-\log p_{\mathcal{F}}(y)) + \sum_y p_{\mathcal{F}}(y) \log \frac{p_{\mathcal{F}}(y)}{q(y)} \\ &= H(Y \mid \mathcal{F}) + D_{\mathrm{KL}}(p_{\mathcal{F}} \| q). \end{aligned}$$

Taking expectation and minimizing over q gives

$$R_{\log}(\mathcal{F}) = H(Y \mid \mathcal{F}).$$

Likewise,

$$R_{\log}(\mathcal{G}) = H(Y \mid \mathcal{G}).$$

By definition of conditional mutual information between Y and the additional information $\mathcal{G}$ given $\mathcal{F}$,

$$I(Y;\mathcal{G} \mid \mathcal{F}) = H(Y \mid \mathcal{F}) - H(Y \mid \mathcal{G}).$$

The conditional-KL form follows from the tower property:

$$\begin{aligned}
&\mathbb{E}\left[D_{\mathrm{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{G}) \| \mathbb{P}(Y \in \cdot \mid \mathcal{F}))\right] \\
&\quad = \mathbb{E}\left[\sum_y p_{\mathcal{G}}(y) \log p_{\mathcal{G}}(y) - \sum_y p_{\mathcal{G}}(y) \log p_{\mathcal{F}}(y)\right] \\
&\quad = -H(Y \mid \mathcal{G}) - \mathbb{E}\left[\sum_y p_{\mathcal{F}}(y) \log p_{\mathcal{F}}(y)\right] \\
&\quad = H(Y \mid \mathcal{F}) - H(Y \mid \mathcal{G}).
\end{aligned}$$

Combining proves the claim. □

Corollary J.4 (Binary distinction value, formal version of Corollary 3.3)**.** *Let $U \in \{0,1\}$ and $\mathcal{G} = \mathcal{F} \vee \sigma(U)$. Under log loss,*

$$\begin{aligned}
R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) &= I(Y;U \mid \mathcal{F}) \\
&= \mathbb{E}\left[\sum_{u \in \{0,1\}} \mathbb{P}(U = u \mid \mathcal{F}) D_{\mathrm{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{F}, U = u) \| \mathbb{P}(Y \in \cdot \mid \mathcal{F}))\right].
\end{aligned}$$

Proof. Apply Theorem J.3 with $\mathcal{G} = \mathcal{F} \vee \sigma(U)$. Conditioning on $\mathcal{F} \vee \sigma(U)$ is the same as conditioning on $\mathcal{F}$ and the binary value of U, yielding the displayed expansion over $u \in \{0,1\}$. □

Theorem J.5 (Squared-loss value, formal version of Theorem 3.4)**.** *Let $Y \in L^2(\Omega;\mathbb{R}^d)$ and let $L(Y,a) = \|Y - a\|_2^2$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_2(\mathcal{F}) - R_2(\mathcal{G}) = \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].$$

Proof. The optimal $\mathcal{F}$-measurable predictor is $\mathbb{E}[Y \mid \mathcal{F}]$, and the optimal $\mathcal{G}$-measurable predictor is $\mathbb{E}[Y \mid \mathcal{G}]$. Let

$$\varepsilon = Y - \mathbb{E}[Y \mid \mathcal{G}], \qquad \delta = \mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}].$$

Then

$$Y - \mathbb{E}[Y \mid \mathcal{F}] = \varepsilon + \delta.$$

Since δ is $\mathcal{G}$-measurable and $\mathbb{E}[\varepsilon \mid \mathcal{G}] = 0$,

$$\mathbb{E}[\langle \varepsilon, \delta \rangle \mid \mathcal{G}] = \langle \mathbb{E}[\varepsilon \mid \mathcal{G}], \delta \rangle = 0.$$

Thus

$$\begin{aligned}
\mathbb{E}\,\|Y - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2 &= \mathbb{E}\,\|\varepsilon + \delta\|_2^2 \\
&= \mathbb{E}\,\|\varepsilon\|_2^2 + \mathbb{E}\,\|\delta\|_2^2 \\
&= R_2(\mathcal{G}) + \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].
\end{aligned}$$

Rearranging proves the identity. □

Theorem J.6 (Minimal loss-sufficient refinement, formal version of Theorem 3.5)**.** *Let $\mathcal{F} \subseteq \mathcal{G}$ and assume the $\mathcal{G}$-Bayes action $a^\star_{\mathcal{G}}$ exists and is almost surely unique. Up to null sets, the smallest sigma-algebra $\mathcal{H}$ satisfying*

$$\mathcal{F} \subseteq \mathcal{H} \subseteq \mathcal{G}, \qquad R_L(\mathcal{H}) = R_L(\mathcal{G})$$

is $\mathcal{F} \vee \sigma(a^\star_{\mathcal{G}})$. For finite Y under log loss, the corresponding smallest sufficient refinement is $\mathcal{F} \vee \sigma(p_{\mathcal{G}})$, where $p_{\mathcal{G}}(\cdot) = \mathbb{P}(Y \in \cdot \mid \mathcal{G})$.

Proof. The action $a^\star_{\mathcal{G}}$ is measurable with respect to $\mathcal{F} \vee \sigma(a^\star_{\mathcal{G}})$, so this refinement attains $R_L(\mathcal{G})$. Let $\mathcal{H}$ be any intermediate sigma-algebra with $R_L(\mathcal{H}) = R_L(\mathcal{G})$. Since $\mathcal{H} \subseteq \mathcal{G}$ and the $\mathcal{G}$-Bayes action is almost surely unique, Theorem J.1 implies that $a^\star_{\mathcal{G}}$ has an $\mathcal{H}$-measurable version. Therefore $\sigma(a^\star_{\mathcal{G}}) \subseteq \mathcal{H}$ up to null sets, and because $\mathcal{F} \subseteq \mathcal{H}$,

$$\mathcal{F} \vee \sigma(a^\star_{\mathcal{G}}) \subseteq \mathcal{H}$$

up to null sets. Under log loss, the Bayes action is the conditional law $p_{\mathcal{G}}$, giving the stated form. □

Proposition J.7 (Finite squared-loss split identity)**.** *Let $B = B_0 \cup B_1$ be a disjoint split of a finite sample set. With empirical means $\hat{\mu}_B, \hat{\mu}_{B_0}, \hat{\mu}_{B_1}$ and* SSE *as in Section 3,*

$$\mathrm{SSE}(B) - \mathrm{SSE}(B_0) - \mathrm{SSE}(B_1) = |B_0| \, \|\hat{\mu}_{B_0} - \hat{\mu}_B\|_2^2 + |B_1| \, \|\hat{\mu}_{B_1} - \hat{\mu}_B\|_2^2 \,.$$

Proof. For each child B_b,

$$\begin{aligned} \sum_{j \in B_b} \|y_j - \hat{\mu}_B\|_2^2 &= \sum_{j \in B_b} \|y_j - \hat{\mu}_{B_b} + \hat{\mu}_{B_b} - \hat{\mu}_B\|_2^2 \\ &= \sum_{j \in B_b} \|y_j - \hat{\mu}_{B_b}\|_2^2 + |B_b| \, \|\hat{\mu}_{B_b} - \hat{\mu}_B\|_2^2 \,, \end{aligned}$$

because

$$\sum_{j \in B_b} (y_j - \hat{\mu}_{B_b}) = 0.$$

Summing over $b \in \{0, 1\}$ gives the result. □

K DGM Algorithm and Projection-Bridge Proofs

This appendix combines elementary finite graph/partition arguments with standard concentration and linear-algebra tools: Hoeffding sample-split validation bounds, nearest-neighbor indexing costs, Moore-Penrose projections, and ridge/Tikhonov least squares [Hoe63, DGL96, IM98, JDJ19, Pen55, TA77, HK70].

Theorem K.1 (Anchor edge separation, formal version of Theorem 4.1)**.** *Let $a_i, a_j \in \mathbb{R}^d$ with $a_i \neq a_j$. Define*

$$u = \frac{a_i - a_j}{\|a_i - a_j\|_2}, \qquad b = \frac{1}{2} u^\top (a_i + a_j).$$

Then

$$u^\top a_i - b = \frac{1}{2} \|a_i - a_j\|_2 \,, \qquad u^\top a_j - b = -\frac{1}{2} \|a_i - a_j\|_2 \,.$$

Proof. Compute

$$\begin{aligned} u^\top a_i - b &= u^\top a_i - \frac{1}{2}u^\top a_i - \frac{1}{2}u^\top a_j \\ &= \frac{1}{2}u^\top (a_i - a_j) \\ &= \frac{1}{2}\left\|a_i - a_j\right\|_2 . \end{aligned}$$

Similarly,

$$\begin{aligned} u^\top a_j - b &= u^\top a_j - \frac{1}{2}u^\top a_i - \frac{1}{2}u^\top a_j \\ &= -\frac{1}{2}u^\top (a_i - a_j) \\ &= -\frac{1}{2}\left\|a_i - a_j\right\|_2 . \end{aligned}$$

□

Theorem K.2 (DGM graph generates a finite information structure, formal version of Theorem 4.2)**.** *Let S be a finite set of binary generators $g_s : \mathsf{H} \to \{0, 1\}$. Define*

$$\sigma_S(h) = \big(g_s(h)\big)_{s \in S}.$$

The relation $h \sim_S h'$ iff $\sigma_S(h) = \sigma_S(h')$ is an equivalence relation. Its nonempty equivalence classes form a finite partition $\mathcal{P}_S^G$ of H. If a new generator g is added, then

$$\mathcal{P}_{S\cup\{g\}}^G = \mathcal{P}_S^G \vee \sigma(g),$$

where the right side denotes cutting each atom of $\mathcal{P}_S^G$ by $g^{-1}(1)$ and $g^{-1}(0)$ and keeping nonempty pieces.

Proof. Equality of signatures is reflexive, symmetric, and transitive, so $\sim_S$ is an equivalence relation. Equivalence classes are disjoint, nonempty, and cover H, hence they form a partition. Since S is finite, there are at most $2^{|S|}$ signatures, so the partition is finite.

For the update, two points have the same signature under $S \cup \{g\}$ exactly when they have the same old S-signature and the same value of g. Therefore each new equivalence class is an old atom intersected with either $g^{-1}(1)$ or $g^{-1}(0)$, with empty intersections discarded. This is precisely $\mathcal{P}_S^G \vee \sigma(g)$. □

Theorem K.3 (Anchor-based hard routing is graph-plus-anchor-order measurable, formal version of Theorem 4.3)**.** *Let E_t be finite and let $a_1, \ldots, a_{N_t} \in \mathbb{R}^d$ be finite frozen anchors with deterministic tie-breaking. Let $\mathcal{F}_t^{\text{edge}} = \sigma(g_e : e \in E_t)$ and let $\mathcal{F}_t^{\text{anchor-order}}$ be generated by all pairwise anchor-distance comparison events*

$$\{h : \|h - a_i\|_2^2 \le \|h - a_j\|_2^2\}.$$

The exact hard-route selected concept index, defined as the nearest eligible anchor with fallback to the nearest anchor, is measurable with respect to $\mathcal{F}_t^{\text{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}$.

Proof. For each concept j, eligibility is an intersection of finitely many edge-bit events of the form

$$\{h : g_e(h) = 1\} \qquad \text{or} \qquad \{h : g_e(h) = 0\},$$

depending on j's recorded side of edge e. Hence the eligible set for j is $\mathcal{F}_t^{\text{edge}}$-measurable.

The anchor-order sigma-algebra determines the complete order of all anchors under the fixed tie-breaking rule. For any subset $R \subseteq \{1, \ldots, N_t\}$, the event that concept j is the nearest anchor in R is a finite intersection of pairwise anchor-comparison events, hence belongs to $\mathcal{F}_t^{\text{anchor-order}}$. The selected hard-route index is obtained by combining finitely many eligibility events with these nearest-in-subset events. Therefore the inverse image of each selected index is a finite union of intersections of sets from $\mathcal{F}_t^{\text{edge}}$ and $\mathcal{F}_t^{\text{anchor-order}}$, and belongs to their join. □

Theorem K.4 (Soft routing approaches hard routing, formal version of Theorem 4.5)**.** *Let N be a finite candidate set and let*

$$\alpha_j = \frac{\exp(-d_j/T)\chi_j}{\sum_{r \in N} \exp(-d_r/T)\chi_r}.$$

If $j^\star_{\text{hard}} \notin N$, where $j^\star_{\text{hard}}$ is the exact hard-route selected concept, then this sparse soft read gives no guarantee for the omitted hard-route concept. Otherwise, assume $j^\star = j^\star_{\text{hard}} \in N$. If $j^\star$ is the unique candidate with $d_j - d_{j^\star} \geq \delta > 0$ for all $j \neq j^\star$ and $\chi_j = \chi_{j^\star} = 1$, then

$$1 - \alpha_{j^\star} \leq (|N| - 1)\exp(-\delta/T).$$

Moreover, suppose $\log \chi_j$ contains a term

$$\frac{\beta_e}{\sqrt{D_j}} \log \operatorname{sigmoid}\left(\frac{m_j}{\tau_e}\right)$$

with $\beta_e > 0$ and finite $D_j \geq 1$, and candidate j violates that edge with $m_j \leq -\delta_e < 0$. Then $\chi_j \leq \exp(-\beta_e \delta_e/(\sqrt{D_j}\tau_e))$.

For a wrong-side set $W \subseteq N$, suppose every $j \in W$ violates some incident structural edge whose weight is at least $\beta_{\min} > 0$, whose signed margin is at most $-\delta_e$, and whose incident degree is at most $D_{\max}$. If at least one eligible reference candidate r has $\chi_r \geq \chi_{\min} > 0$, and the distance score ratio between every $j \in W$ and r is bounded by C, then the total wrong-side routing mass is at most

$$\frac{C\,|N|}{\chi_{\min}} \exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right).$$

Proof. If $j^\star_{\text{hard}} \notin N$, then the soft normalization assigns weight only to candidates in N and the omitted hard-route concept has no route to receive weight. This is the candidate-recall caveat.

For the nearest-reference statement,

$$\frac{\sum_{j \neq j^\star} \exp(-d_j/T)}{\exp(-d_{j^\star}/T)} = \sum_{j \neq j^\star} \exp(-(d_j - d_{j^\star})/T) \leq (|N| - 1)\exp(-\delta/T).$$

Since

$$1 - \alpha_{j^\star} = \frac{\sum_{j \neq j^\star} \exp(-d_j/T)}{\exp(-d_{j^\star}/T) + \sum_{j \neq j^\star} \exp(-d_j/T)},$$

the same upper bound follows.

For an edge violation, set $a = \delta_e/\tau_e > 0$. Since

$$\text{sigmoid}(-a) = \frac{1}{1+\exp(a)} \le \exp(-a),$$

the corresponding log-compatibility contribution is at most $-\beta_e\delta_e/(\sqrt{D_j}\tau_e)$, and therefore $\chi_j \le \exp(-\beta_e\delta_e/(\sqrt{D_j}\tau_e))$. If W is the wrong-side set and r is an eligible candidate whose compatibility satisfies $\chi_r \ge \chi_{\min}$, the uniform wrong-side assumptions give, for every $j \in W$,

$$\chi_j \le \exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right).$$

Combining this with the assumed distance-score ratio bound gives

$$\begin{aligned}\sum_{j\in W}\alpha_j &\le \sum_{j\in W}\frac{C}{\chi_{\min}}\exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right)\\ &\le \frac{C\,|N|}{\chi_{\min}}\exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right).\end{aligned}$$

□

Theorem K.5 (Refine-or-repair objective descent, formal version of Theorem 4.7)**.** *Let*

$$J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}) = \widehat{R}(\mathcal{S};\mathcal{B}_t^{\text{score}}) + \lambda_{\text{concept}}N_{\text{concept}}(\mathcal{S}) + \lambda_{\text{edge}}\,|E(\mathcal{S})|$$

with nonnegative penalties on a fixed scoring buffer. If a repair step is committed only after the guarded test

$$\widehat{R}(\mathcal{S}_{t+1};\mathcal{B}_t^{\text{score}}) \le \widehat{R}(\mathcal{S}_t;\mathcal{B}_t^{\text{score}}),$$

and leaves anchors, edges, and concept count unchanged, then $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. If a refinement creates one concept and q edges and is accepted only when

$$\widehat{\Delta}_{\text{refine}} > \max\{\widehat{\Delta}_{\text{repair}}, 0\} + \tau + 2\epsilon_m,$$

where $\widehat{\Delta}_{\text{repair}} \ge 0$ is the empirical repair gain and

$$\widehat{\Delta}_{\text{refine}} = \widehat{R}(\mathcal{S}_t;\mathcal{B}_t^{\text{score}}) - \widehat{R}(\mathcal{S}_{t+1};\mathcal{B}_t^{\text{score}}) - \lambda_{\text{concept}} - \lambda_{\text{edge}}q,$$

then

$$J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_{t+1}) < J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_t) - \widehat{\Delta}_{\text{repair}} - \tau - 2\epsilon_m.$$

In particular, $J_{\mathcal{B}_t^{\text{score}}}$ decreases by more than $\tau + 2\epsilon_m$.

Proof. For repair, N_{concept} and $|E|$ are unchanged. The empirical-risk term on $\mathcal{B}_t^{\text{score}}$ is no larger by the guard, so $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. The identity candidate is always feasible, so the guarded rule can decline harmful centroid or memory movement instead of committing it.

For refinement,

$$\begin{aligned}J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_t) - J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_{t+1}) &= \widehat{R}(\mathcal{S}_t;\mathcal{B}_t^{\text{score}}) - \widehat{R}(\mathcal{S}_{t+1};\mathcal{B}_t^{\text{score}}) - \lambda_{\text{concept}} - \lambda_{\text{edge}}q\\ &= \widehat{\Delta}_{\text{refine}}\\ &> \widehat{\Delta}_{\text{repair}} + \tau + 2\epsilon_m.\end{aligned}$$

Rearranging gives the claim. □

The theorem is local to the fixed scoring buffer used in its statement. If the scoring buffer changes over time, these inequalities alone do not define a common global descent potential.

Theorem K.6 (Statistically safe adaptive refinement, formal version of Theorem 4.8)**.** *Assume losses lie in* $[0, 1]$*. Let* $(\mathfrak{H}_t)$ *be the filtration containing the past stream, current DGM state, and all proposal-buffer information available before scoring at time* t*. Let* $\mathcal{C}_t$ *be a finite candidate class predictable with respect to* $\mathfrak{H}_t$*, and let* $\mathcal{B}_t^{\text{score}}$ *contain* m_t *samples independent of* $\mathfrak{H}_t$ *conditional on the current scoring distribution. For candidate* g*, let* $\widehat{\Delta}_t(g)$ *be the empirical gain on* $\mathcal{B}_t^{\text{score}}$ *and let* $\Delta_t(g)$ *be the corresponding population gain. Define*

$$\epsilon_t = \sqrt{\frac{\log(4t^2(|\mathcal{C}_t| + 1)/\delta)}{2m_t}}.$$

Then, with probability at least $1 - \delta$*, every time* t *and candidate* $g \in \mathcal{C}_t$ *satisfy*

$$\left|\widehat{\Delta}_t(g) - \Delta_t(g)\right| \leq 2\epsilon_t.$$

Consequently, if

$$\widehat{\Delta}_t(g) > \lambda \operatorname{cost}(g) + \tau + 2\epsilon_t,$$

then on the same event

$$\Delta_t(g) > \lambda \operatorname{cost}(g) + \tau.$$

Proof. Condition on $\mathfrak{H}_t$. The candidate class is then fixed, and the scoring buffer is independent of the proposal mechanism by assumption. Apply Hoeffding's inequality to the empirical risk of the old state and to the empirical risk of each candidate-refined state. For any fixed state in this finite collection,

$$\mathbb{P}\left(\left|\widehat{R} - R\right| > \epsilon\right) \leq 2\exp(-2m\epsilon^2).$$

With $m = m_t$ and $\epsilon = \epsilon_t$, the failure probability at time t is at most $\delta/(2t^2)$ after a union bound over the old state and the $|\mathcal{C}_t|$ candidate states. Since $\sum_{t\geq 1} t^{-2} < 2$, the displayed constant $4t^2$ gives total failure probability at most δ over all times. On this event, each gain is a difference of two risks, so empirical and population gain differ by at most $2\epsilon_t$. The implication follows by subtracting this deviation from the empirical gain lower bound. □

Corollary K.7 (Bounded language-model refinement score, formal version of Corollary 4.9)**.** *For language modeling, let*

$$\ell_{\text{clip}}(x_{\leq t}, x_{t+1}) = \frac{1}{L_{\max}} \min\{-\log p(x_{t+1} \mid x_{\leq t}), L_{\max}\}$$

for a fixed $L_{\max} > 0$*. If the scoring buffer, empirical gain, and population gain in Theorem K.6 are computed using* ℓ_{clip}*, then the theorem applies to language-model refinement decisions made with this clipped score. In particular, an accepted refinement has positive clipped population gain after paying complexity cost and margin* τ *with probability at least* $1 - \delta$*.*

Proof. The clipped score satisfies $0 \leq \ell_{\text{clip}} \leq 1$. Substituting this bounded loss into Theorem K.6 gives the claim directly. The statement is only about the clipped score; raw negative log likelihood is unbounded and would require a separate concentration assumption. □

Theorem K.8 (Anchor-stable DGM information structure, formal version of Theorem 4.10)**.** *Let*

$$\mathcal{F}_t^{\mathrm{DGM}} = \sigma(g_e : e \in E_t) \vee \sigma\Big(\{h : \|h - a_i\|_2^2 \le \|h - a_j\|_2^2\} : 1 \le i, j \le N_t\Big).$$

Suppose every edge generator $g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}$ is computed from frozen endpoint anchors a_i, a_j. Any repair step that changes only centroids c_j, local memories M_j, and counts n_j, while keeping anchors and edges fixed, leaves $\mathcal{F}_t^{\mathrm{DGM}}$ unchanged. If a refinement adds a new anchor and new edge generators, then $\mathcal{F}_{t+1}^{\mathrm{DGM}}$ is $\mathcal{F}_t^{\mathrm{DGM}}$ joined with the new anchor-order events and the new edge generators.

Proof. For an edge $e = (i, j)$, both

$$u_e = \frac{a_i - a_j}{\|a_i - a_j\|_2}, \qquad b_e = \frac{1}{2} u_e^\top (a_i + a_j)$$

are deterministic functions of the frozen anchors. A repair step does not change these anchors, so it does not change u_e, b_e, or g_e. It also does not change any anchor-distance comparison event because all anchors are fixed. Hence both sigma-algebras in the join defining $\mathcal{F}_t^{\mathrm{DGM}}$ are unchanged.

If a refinement adds anchor a_{new} and edge generators E_{new}, the new edge sigma-algebra is $\sigma(g_e : e \in E_t) \vee \sigma(g_e : e \in E_{\mathrm{new}})$, and the new anchor-order sigma-algebra is the old anchor-order sigma-algebra joined with the pairwise comparisons involving a_{new}. This is exactly the stated join. □

Corollary K.9 (Finite number of accepted refinements, formal version of Corollary 4.11)**.** *Let Φ be a common potential with $\Phi(\mathcal{S}) \ge 0$ for all reachable states. If each accepted refinement decreases Φ by at least $\tau > 0$, then the number of accepted refinements is at most $\Phi(\mathcal{S}_0)/\tau$.*

Proof. After m accepted refinements,

$$\Phi(\mathcal{S}_m) \le \Phi(\mathcal{S}_0) - m\tau.$$

Since $\Phi(\mathcal{S}_m) \ge 0$, we have $m \le \Phi(\mathcal{S}_0)/\tau$. □

Proposition K.10 (Trees as generalized hard-route DGMs, formal version of Proposition E.1)**.** *Let $\mathcal{T}$ be a binary decision tree whose internal nodes compute binary distinctions $g_\nu(h) \in \{0, 1\}$ and whose leaves have predictors M_ℓ. Construct a generalized hard-route DGM with one concept c_ℓ and memory M_ℓ for each leaf ℓ, where the generalized model permits arbitrary finite binary eligibility constraints in addition to pairwise distinction edges. For each leaf, add hard eligibility constraints equal to the signed distinctions along the root-to-leaf path. If the generalized hard route selects the unique eligible leaf concept, then its prediction equals the tree prediction for every h.*

Proof. For any input h, the tree follows exactly one root-to-leaf path. The leaf reached by the tree is the unique leaf whose path constraints are all satisfied by h. By construction, the corresponding generalized hard-route concept has exactly those eligibility constraints, and all other leaf concepts violate at least one path constraint at the first node where their path differs from the tree path. Therefore the generalized hard route selects the same leaf and applies the same memory M_ℓ. □

Theorem K.11 (DGM data-structure bound, formal version of Theorem E.2)**.** *Assume DGM has N_t concepts in $\mathbb{R}^d$, exact centroid and anchor retrieval, candidate set $N(h)$, selected incident edge-gate count $Q(h) = \sum_{j \in N(h)} |\mathrm{Inc}_t(j)|$, and local-memory evaluation cost C_{local}. Then one prediction costs $O(N_t d + Q(h) + |N(h)|\, d + C_{\mathrm{local}})$. One observation that repairs a selected concept costs this*

plus the guarded local repair scoring cost. One observation that creates a new anchor-centroid pair and q distinction edges costs an additional $O(d+qd)$. If each concept stores an anchor, a centroid, and a sparse categorical memory with average support $\bar{s}$, and each edge stores a linear distinction in $\mathbb{R}^d$, then the runtime state uses $O(N_t(2d+\bar{s})+|E_t|\,d)$ memory.

Proof. A distance computation between $h \in \mathbb{R}^d$ and one concept center costs $O(d)$. Exact centroid and anchor retrieval over N_t concepts therefore costs

$$O(N_t d).$$

Graph-neighbor expansion has cost proportional to the incident lists touched and is absorbed into selected edge-gate bookkeeping. After the candidate set $N(h)$ is selected, evaluating the incident edge gates costs $O(Q(h))$. Computing sparse routing weights, distances within the selected set, and evaluating selected local memories costs

$$O(|N(h)|\,d + C_{\text{local}}).$$

Thus prediction costs

$$O(N_t d + Q(h) + |N(h)|\,d + C_{\text{local}}).$$

Observation begins with the same retrieval computation. If it repairs a selected concept, it adds the cost of evaluating and scoring the proposed local repair candidates on the scoring buffer. If it creates a new anchor-centroid pair, storing the pair costs $O(d)$. For each new edge, computing and storing the linear distinction vector costs $O(d)$, so q edges cost

$$O(qd).$$

The total additional cost for refinement is therefore $O(d+qd)$.

For memory, each concept stores $a_j, c_j \in \mathbb{R}^d$ and a sparse categorical memory of average support $\bar{s}$, giving

$$O(N_t(2d+\bar{s})).$$

Each edge stores one distinction vector and threshold, giving

$$O(|E_t|\,d).$$

Adding the two terms proves the claimed bound. □

Theorem K.12 (DGM layer cost, formal version of Theorem E.3)**.** *Consider a DGM adapter extension with hidden dimension d, query dimension d_q, local rank r, candidate count $|N|$, selected incident edge-gate count Q_N, and concept-bank query cost $C_{\text{ret}}(N_t, d_q)$. For one token, the read overhead is*

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N + |N|\,dr).$$

If the write path is detached and uses rank-one local repair, the write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N + |N|\,(d_q + dr)).$$

For a block of length L, backpropagation through the slow parameters does not require storing the full DGM state history $\mathcal{S}_1, \ldots, \mathcal{S}_L$.

Proof. For a token hidden state $h \in \mathbb{R}^d$, the query computation

$$q = Qh$$

costs $O(dd_q)$. The retrieval cost is $C_{\text{ret}}(N_t, d_q)$ by definition. Evaluating selected incident edge gates costs $O(Q_N)$.

For a selected concept j, the rank-r adapter read is

$$W_j K h.$$

Multiplying one selected $W_j \in \mathbb{R}^{d\times r}$ by a rank-r key costs $O(dr)$. Therefore $|N|$ selected adapters cost

$$O(|N|\, dr).$$

The routing weights add lower-order top-k normalization cost, absorbed into the same bound. This proves the read overhead.

The write path first computes the same query, retrieval, and edge gates. It then updates selected concept statistics and selected local memories. Concept bookkeeping costs $O(d_q)$ per selected concept. A rank-one local repair of a rank-r or low-rank sufficient-statistic memory costs $O(dr)$ per selected memory. Hence the write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N + |N|\,(d_q + dr)).$$

Finally, the update rule is applied as

$$\text{stopgrad}(\mathcal{S}_{t+1}) = \text{DGMObserve}(\text{stopgrad}(\mathcal{S}_t), h_t, x_{t+1}).$$

Thus the computational graph for the loss contains the selected read computation, but not the chain of state-update maps from $\mathcal{S}_1$ through $\mathcal{S}_L$. Therefore full DGM state history is not required for backpropagation through the slow parameters. □

Theorem K.13 (One-hot memory is partition-measurable, formal version of Theorem H.1)**.** *Let $\mathcal{P} = \{B_1, \ldots, B_n\}$ be a finite partition and let $\phi_{\mathcal{P}}(\omega) = e_i$ for $\omega \in B_i$. For every $W \in \mathbb{R}^{m\times n}$, the map $f_W(\omega) = W\phi_{\mathcal{P}}(\omega)$ is $\mathcal{P}$-measurable. Conversely, every $\mathcal{P}$-measurable map $f : \Omega \to \mathbb{R}^m$ is equal to f_W for some $W \in \mathbb{R}^{m\times n}$.*

Proof. If $\omega, \omega' \in B_i$, then

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

Thus

$$f_W(\omega) = We_i = f_W(\omega'),$$

so f_W is constant on each atom and is $\mathcal{P}$-measurable.

Conversely, if f is $\mathcal{P}$-measurable, it is constant on each B_i. Let w_i be this constant value on B_i, and define

$$W = [w_1, \ldots, w_n].$$

Then for $\omega \in B_i$,

$$W\phi_{\mathcal{P}}(\omega) = We_i = w_i = f(\omega).$$

□

Theorem K.14 (CPM conflict equals missing distinction, formal version of Theorem H.2)**.** *Let* $\mathcal{P} = \{B_1, \ldots, B_n\}$ *and let* $\omega, \omega' \in B_i$*. Suppose* $v(\omega) \neq v(\omega')$*. Let* $k = \phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i$*. If a memory* W *has already committed* $We_i = v(\omega)$*, then the evidence* $We_i = v(\omega')$ *has innovation* $z = 0$ *relative to the old key span and residual* $e \neq 0$*.*

Proof. Since $\omega, \omega' \in B_i$,

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

The old key span contains e_i. Therefore the innovation of the new key $k = e_i$ relative to the old key matrix $K = e_i$ is

$$z = (I - KK^{\dagger})k = (I - e_i e_i^{\top})e_i = 0.$$

The new residual is

$$e = v(\omega') - We_i = v(\omega') - v(\omega).$$

Since $v(\omega') \neq v(\omega)$, we have $e \neq 0$. □